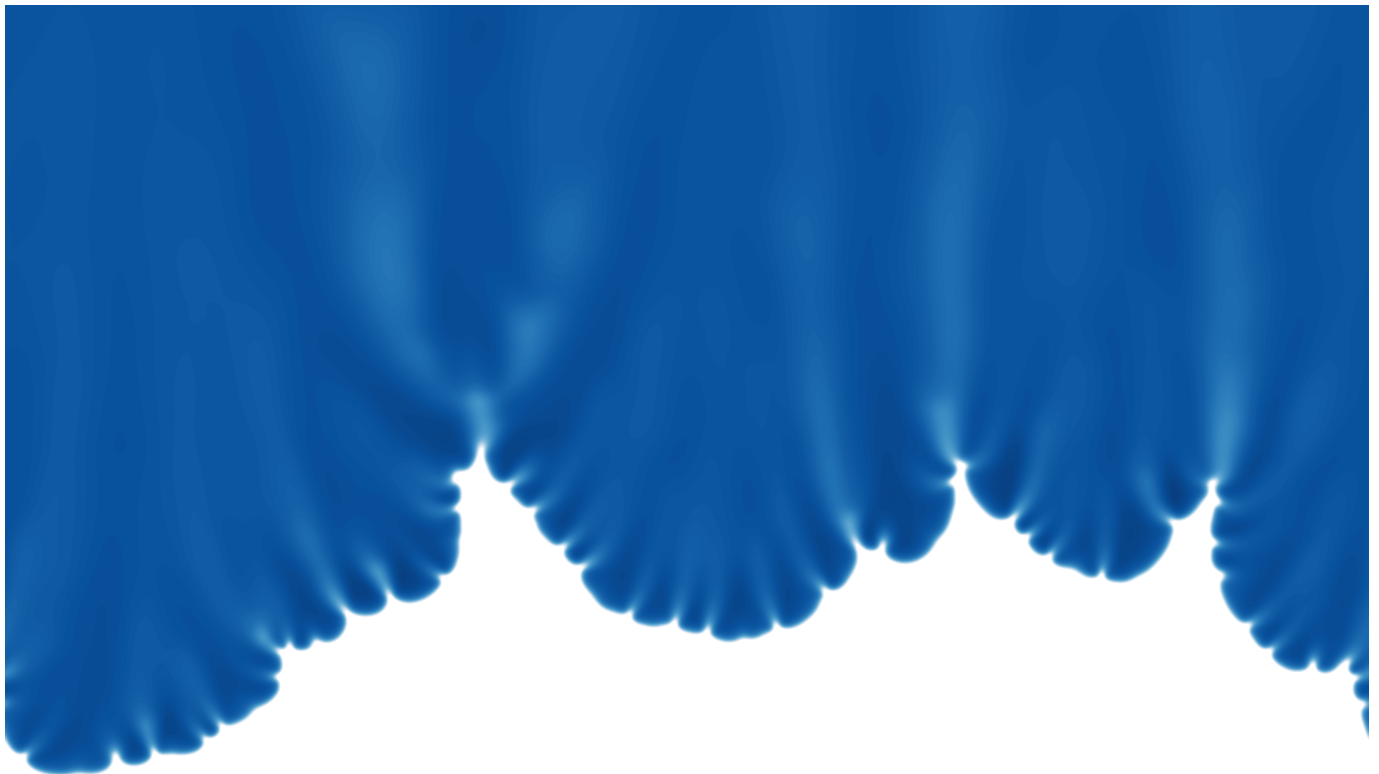


Manual to compute Flamelet Generated Manifold tables

Author: Antonio Masucci^{*a,1}

^a*Università degli Studi di Cagliari*

This manuscript was compiled on July 3, 2026



^{*}antonio.masucci.1997@gmail.com (A. Masucci)

CONTENTS

1	Introduction	3
2	Python Scripts	3
2.1	scaleYc.py	3
2.2	createFGMTables.py	8
2.3	extendFGMTables.py	19
2.4	integrateFGMTables.py	28

1. INTRODUCTION

The accurate and efficient simulation of reactive flows remains a major challenge in computational fluid dynamics (CFD), particularly when detailed chemical kinetics are involved. High fidelity simulations of reacting flows can be computationally expensive when detailed chemistry is taken into account. To overcome this limitation, tabulated chemistry methods have been widely adopted, among which the Flamelet Generated Manifold (FGM) approach represents a robust and efficient framework.

The FGM method relies on the precomputation of thermochemical quantities in a reduced-dimensional manifold, which is then stored in the form of look-up tables. These tables map a set of controlling variables to relevant thermochemical properties such as temperature, species mass fractions, and reaction rates. During the CFD simulation, these quantities are retrieved through interpolation, significantly reducing the computational cost compared to solving detailed chemistry on-the-fly.

This document describes the methodology adopted for the generation of FGM tables used in the present work. In particular, the effects of preferential diffusion, arising from the high diffusivity of hydrogen compared to other species, are explicitly considered following the formulation of Perez et al. [2] Additionally, thermal diffusion (Soret effect), which induces species transport driven by temperature gradients, is incorporated in the tabulation process. These effects are known to significantly influence flame structure, local equivalence ratio, and overall combustion characteristics, especially in lean hydrogen-air mixtures. The resulting FGM tables are designed to be integrated into an OpenFOAM-based solver, where they are accessed through a dedicated interpolation routine. The overall goal is to provide a reliable and computationally efficient representation of complex chemistry, suitable for high-fidelity simulations of reactive flows, including hydrogen-air combustion systems.

2. PYTHON SCRIPTS

The FGM tables are created using Python scripts, with Cantera employed as a one-dimensional solver to resolve the detailed chemistry. To generate the final table to be used by the OpenFOAM solver, four separate scripts must be executed, named as follows:

1. `scaleYc.py`
2. `createFGMTables.py`
3. `extendFGMTables.py`
4. `integrateFGMTables.py`

A description of each script is given below.

2.1. `scaleYc.py`

This script generates the file required to scale the progress variable, defined as:

$$C = \frac{Y_c - Y_{c,u}(Z)}{Y_{c,b}(Z) - Y_{c,u}(Z)} \quad (1)$$

This scaling is required because the flamelet variables ζ_i are parameterized as functions of the mixture fraction Z and the scaled progress variable C .

```
# -*- coding: utf-8 -*-
"""
CREATE Yc Scaling function - Yc,b = f(Z) to scale C = Yc/Yc,b(Z)
Parallel version - each flamelet (phi) is solved in a separate worker process.
"""

import pandas as pd
```

```

import cantera as ct
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import interp1d
import os
import matplotlib as mpl
import ctypes
from multiprocessing import Pool, Value
import modules.functions as m

mpl.rcParams['text.usetex'] = True
mpl.rcParams['font.family'] = 'serif'
mpl.rcParams['font.serif'] = ['Computer Modern']

# -----
# CONFIGURATION
# -----

save = True
Soret = False

nameMech = 'SanDiegoNO'
mech      = nameMech + '.yaml'

if Soret:
    path = "./" + nameMech + "Soret/"
else:
    path = "./" + nameMech + "/"

CANTERA_DATA_DIR = "C:/Program Files/Cantera/data"

# Number of parallel workers (leave 1 core free)
N_WORKERS = max(1, os.cpu_count() - 1)

```

All required libraries are first imported. Then, two switches, `save` and `Soret`, are defined to control whether the final file should be written and whether Soret effects are included in the flamelet computations. The path `CANTERA_DATA_DIR` specifies the directory where the reaction mechanisms are stored. Within the script, Cantera is executed in parallel to improve performance, utilizing all available CPU cores except one.

```

# -----
# DISCRETIZATION
# -----

nPointsZ = 300
a         = 3          # controls curvature
s         = np.linspace(0, 1, nPointsZ)
phiLean   = 0.28
phiRich   = 5.0
phiVector = phiLean + (phiRich - phiLean) * (np.exp(a * s) - 1) / (np.exp(a) - 1)

# -----
# SHARED COUNTER (tracks progress across worker processes)

```

```

# -----
def _init_counter(counter, total):
    global _counter, _total
    _counter = counter
    _total = total

# -----
# WORKER FUNCTION (one call per phi, runs in a separate process)
# -----
def runScaleYcFlamelet(args):
    """
    Solve one premixed flamelet and return the mixture fraction Z and the
    maximum progress-variable value YcMax at that phi.

    Must be a top-level function so multiprocessing can pickle it.
    All imports are done inside the function to avoid pickling issues with
    Cantera objects.
    """
    i, phi, p, tin, mech, Soret, cantera_dir = args

    import cantera as ct
    import numpy as np
    import modules.functions as m

    ct.add_directory(cantera_dir)

    # -- Solve flamelet -----
    f, gas = m.premixedFlame(p, tin, phi, mech, Soret)

    with _counter.get_lock():
        _counter.value += 1
        done = _counter.value
        pct = done / _total * 100
        print(f" Flamelets completed: {done}/{_total} ({pct:.1f}%)", flush=True)

    # -- Fresh gas object at this phi -----
    gas2 = ct.Solution(mech)
    air = "O2:0.21,N2:0.79"
    gas2.set_equivalence_ratio(phi=phi, fuel="H2:1", oxidizer=air)
    gas2.TPX = tin, p, gas2.X

    YcMax = float(np.max(f.Y[gas2.species_index('H2O')]))
    ZPhi = float(gas2.mixture_fraction(fuel="H2:1", oxidizer=air, element='Bilger'))

    return i, ZPhi, YcMax

```

The mixture-fraction is discretized using `nPointsZ = 300` points. A non-uniform grid is generated through an exponential mapping controlled by the parameter `a`, which increases the resolution toward the rich side of the domain. The equivalence ratio range is defined between `phiLean = 0.28` and `phiRich = 5.0`, and the resulting `phiVector` represents the set of flamelet conditions to be solved.

The helper function `_init_counter(counter, total)` initializes a shared progress counter for multiprocessing. Both the counter and the total number of flamelets are stored as global variables, allowing each worker process to update a common progress indicator during execution.

The function `runScaleYcFlamelet(args)` computes a single premixed flamelet for a given equivalence ratio ϕ , returning the corresponding mixture-fraction value and the maximum unscaled progress variable $Y_{c,b}(Z)$. It is implemented as a top-level function to ensure it can be serialized and used within the multiprocessing framework.

```
# -----
# MAIN
# -----
if __name__ == "__main__":

    ct.add_directory(CANTERA_DATA_DIR)

    p    = ct.one_atm
    tin  = 298.15

    # -- Build argument list for workers -----
    args_list = [
        (i, phi, p, tin, mech, Soret, CANTERA_DATA_DIR)
        for i, phi in enumerate(phiVector)
    ]

    # -- Run flamelets in parallel -----
    print(f"\nRunning {nPointsZ} flamelets on {N_WORKERS} workers...\n")

    counter = Value(ctypes.c_int, 0)
    with Pool(processes=N_WORKERS,
              initializer=_init_counter,
              initargs=(counter, nPointsZ)) as pool:
        rawResults = pool.map(runScaleYcFlamelet, args_list)

    # Sort by original phi index to preserve order
    rawResults.sort(key=lambda x: x[0])

    # -- Unpack results -----
    Z_values = np.array([r[1] for r in rawResults])
    YcMax     = np.array([r[2] for r in rawResults])
    YcMin     = np.zeros(nPointsZ)

    print("\nAll flamelets done. Building scaling table...")

    # -- Interpolate onto uniform Z grid -----
    ZVec = np.linspace(0, 1, 1000)

    ZExt    = np.concatenate(([0], Z_values, [1]))
    YcMaxExt = np.concatenate(([0], YcMax, [0]))

    f        = interp1d(ZExt, YcMaxExt, fill_value='extrapolate')
    YcMaxInt  = f(ZVec)
    YcMinInt  = np.zeros_like(YcMaxInt)
```

```

plt.plot(ZVec, YcMaxInt)
plt.xlabel("Z")
plt.ylabel("$Y_{c,b}$")
plt.title("Scaling function $Y_{c,b}(Z)$")

# -- Save -----
df = pd.DataFrame({
    'Z': ZVec,
    'YcMax': YcMaxInt,
    'YcMin': YcMinInt,
})

if save:
    os.makedirs(nameMech, exist_ok=True)
    df.to_csv(path + 'scaleYc' + nameMech + '.csv', index=False)
    m.scaleYc(ZVec, YcMinInt, YcMaxInt, filename="scalingYcTableSoret")
    print("Scale Yc Function successfully saved in folder ---> " + nameMech + "!")

```

The script is executed within the standard `if name == "main":` guard to ensure correct behavior when using multiprocessing. This block initializes Cantera, sets the thermodynamic operating conditions, launches the flamelet computations in parallel, and post-processes the results to construct the tabulated scaling function $Y_{c,b}(Z)$.

An argument list is then assembled for all equivalence-ratio values contained in `phiVector`. Each entry includes the flamelet index, the corresponding ϕ , and all thermochemical and configuration parameters required by the worker function.

The flamelets are solved in parallel using a process pool with `N_WORKERS` workers. A shared counter tracks global progress and prints the number of completed flamelets during execution. Once all workers have finished, the results are sorted by index to preserve the original ordering of `phiVector`.

The discrete data are then interpolated onto a uniform mixture-fraction grid `ZVec` spanning the interval $[0,1]$. Boundary points at $Z = 0$ and $Z = 1$ are added to ensure a complete and well-defined table. The resulting scaling curve `YcMaxInt` defines the upper envelope $Y_{c,b}(Z)$, while `YcMinInt` remains identically zero in this case, since no water vapor is present in the reactants. The shape of this function is shown in Fig. 1.

In the end, the file `scalingYcTable` is generated in a format that can be read by the OpenFOAM solver using the Python function `scaleYc`.

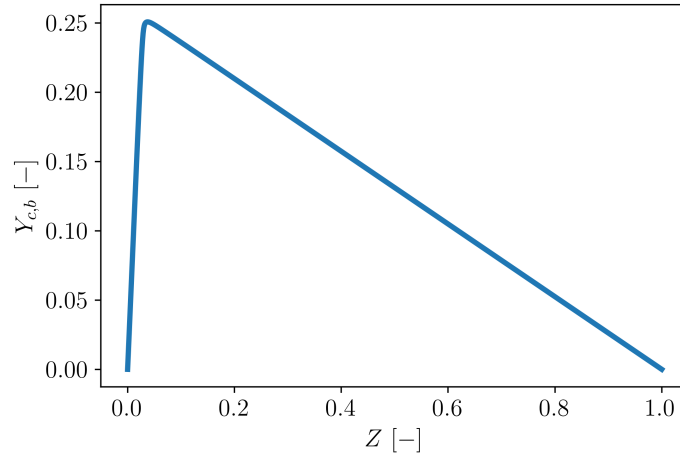


Figure 1. Scaling function $Y_{c,b}(Z)$

2.2. createFGMTables.py

This script is used to compute the flamelets using Cantera, which are subsequently employed to build the look-up table.

First, several switches are defined at the beginning:

```
# -----
# CONFIGURATION
# -----
save          = True
plotTables    = False
Soret         = False

nameMech      = 'SanDiegoNO'
mech          = nameMech + '.yaml'

if Soret:
    path       = "./" + nameMech + "Soret/"
    print("Soret activated!")
else:
    path       = "./" + nameMech + "/"
    print("Soret deactivated!")

CANTERA_DATA_DIR = "C:/Program Files/Cantera/data"

# Number of parallel workers (leave 1 core free)
N_WORKERS = max(1, os.cpu_count() - 1)
```

1. `save`: enables saving the tables computed with `premixedFlame` function and interpolated over the vector C , which is defined later.
2. `plotTables`: controls whether the tables are plotted after the flamelet computations are completed. It is recommended to keep this option disabled and perform visualization separately at the end.
3. `Soret`: enables thermal diffusion effects in the mixture-averaged transport model used by [2]. Depending on this switch, the tables are stored either in the folder `nameMech` or in `"nameMech" + "Soret"`.

The flamelets are computed based on the definition of `phiVector`, which is generated using an exponential distribution. The same mapping is also used to construct the C vector, which defines the interpolation points for the flamelet solutions. This approach is adopted to increase the resolution in the reaction-zone region, where strong gradients are present.

```
# -----
# DISCRETIZATION
# -----
nPointsZ = 300
a         = 3
s         = np.linspace(0, 1, nPointsZ)
phiLean   = 0.28
phiRich   = 5.0
phiVector = phiLean + (phiRich - phiLean) * (np.exp(a * s) - 1) / (np.exp(a) - 1)

nPointsC = 600
a         = -6
x         = np.linspace(0, 1, nPointsC)
c         = (np.exp(a * x) - 1) / (np.exp(a) - 1)
```

As previously done for the computation of $Y_{c,b}(Z)$, the flamelets are also run in parallel. This procedure is identical to the one described in the previous section.

```
# -----
# WORKER FUNCTION (runs in a separate process for each phi)
# -----
# Shared counter initializer – called once per worker process at startup
def _init_counter(counter, total):
    global _counter, _total
    _counter = counter
    _total   = total

def runFlamelet(args):
    """
    Solve one premixed flamelet and return all tabulated data for that phi.
    Must be a top-level function so multiprocessing can pickle it.
    All imports are done inside the function to avoid pickling issues with
    Cantera objects.
    """
    i, phi, p, tin, mech, c, YScal, Soret, cantera_dir = args

    # Re-import inside worker (Cantera objects are not safely picklable)
    import cantera as ct
    import numpy as np
    import modules.functions as m

    ct.add_directory(cantera_dir)

    # -- Solve flamelet -----
    f, gas = m.premixedFlame(p, tin, phi, mech, Soret=Soret)
```

```

with _counter.get_lock():
    _counter.value += 1
    done = _counter.value
pct = done / _total * 100
print(f" Flamelets completed: {done}/{_total} ({pct:.1f}%)", flush=True)
species = gas.species_names

# Fresh gas object at this phi for mixture-fraction calculation
gas2 = ct.Solution(mech)
air = "O2:0.21,N2:0.79"
gas2.set_equivalence_ratio(phi=phi, fuel="H2:1", oxidizer=air)
gas2.TPX = tin, p, gas2.X

ZPhi = gas2.mixture_fraction(fuel="H2:1", oxidizer=air, element='Bilger')
sL = f.velocity[0]
tau = (f.T[-1] - f.T[0]) / f.T[0]
lF = (f.T[-1] - f.T[0]) / np.max(np.gradient(f.T, f.grid))

# -- Bilger mixture fraction along the flame -----
ZBilg = m.BilgerMixtureFraction(gas2, f, mech)

# -- Chemical enthalpy -----
hSpecies = (gas2.standard_enthalpies_RT * ct.gas_constant * gas2.T) /
    ↪ gas2.molecular_weights
hChem = np.dot(hSpecies, f.Y)

# -- Scale progress variable Yc -> C -----
YcMax = np.interp(ZBilg, YScal[:, 0], YScal[:, 1])
cStar = f.Y[gas2.species_index('H2O')] / YcMax
cStar, idx = np.unique(cStar, return_index=True)
cStar = (cStar - cStar.min()) / (cStar.max() - cStar.min())

```

Inside the function `runFlamelet`, the flamelets are computed using the function `premixedFlame`, which takes as input the operating pressure `p` and inlet temperature `tin`, the equivalence ratio `phi`, the reaction mechanism `mech`, and the switch `Soret` to enable thermal diffusion effects.

The mixture fraction is calculated using Bilger's formulation through the function `BilgerMixtureFraction`, which computes Z as:

$$Z = \frac{\frac{1}{2W_H}(Z_H - Z_{H,o}) - \frac{1}{W_O}(Z_O - Z_{O,o})}{\frac{1}{2W_H}(Z_{H,f} - Z_{H,o}) - \frac{1}{W_O}(Z_{O,f} - Z_{O,o})} \quad (2)$$

where superscripts u and b refer to unburnt and burnt mixtures respectively, subscripts f and o indicate fuel and oxidizer streams, and Z_i refers to the elemental mass fractions.

The unscaled progress variable (based on water vapor in this case) is then scaled using the `scaleYc` function defined previously, in order to obtain the vector c . Any duplicate values are removed if present.

```

# Helper: interpolate a 1-D array from cStar -> c
def interp(arr):
    return m.interpExt(c, cStar, arr[idx])

```

```

# -- Build data dictionary -----
data = {
    "x"      : interp(f.grid),
    "T"      : interp(f.T),
    "U"      : interp(f.velocity),
    "rho"    : interp(f.density),
    "Z"      : interp(ZBilg),
    "Yc"     : interp(f.Y[gas2.species_index('H2O')]),
    "c"      : c,
    "Cp"     : interp(f.cp_mass),
    "k"      : interp(f.thermal_conductivity),
    "D"      : interp(f.thermal_conductivity / (f.density * f.cp_mass)),
    "h"      : interp(f.enthalpy_mass),
    "dH"     : interp(hChem),
    "HRR"    : interp(f.heat_release_rate),
    "MMW"    : interp(f.mean_molecular_weight) * 1e-3,
    # scalar flame properties
    "sL"     : sL,
    "tau"    : tau,
    "lF"     : lF,
    "ZPhi"   : ZPhi,
}

for sp in species:
    sp_idx = gas2.species_index(sp)
    data[f"D_{sp}"] = interp(f.mix_diff_coeffs[sp_idx])
    data[f"DSoret_{sp}"] = interp(f.thermal_diff_coeffs[sp_idx])
    data[f"Y_{sp}"] = interp(f.Y[sp_idx])
    data[f"X_{sp}"] = interp(f.X[sp_idx])
    data[f"h_{sp}"] = interp(f.partial_molar_enthalpies[sp_idx] /
        ↪ gas2.molecular_weights[sp_idx])
    data[f"omega_{sp}"] = interp(f.net_production_rates[sp_idx] *
        ↪ gas2.molecular_weights[sp_idx])

data["sourceYcV"] = interp(
    f.net_production_rates[gas2.species_index('H2O')]
    * gas2.molecular_weights[gas2.species_index('H2O')]
    * f.Y[gas2.species_index('H2O')]
)

return i, phi, data

```

The function `interp` is defined to interpolate the flamelet solutions computed with Cantera onto the C vector defined at the beginning of the script.

At this point, everything is set to start.

```

# -----
# MAIN
# -----

```

```

if __name__ == "__main__":

    ct.add_directory(CANTERA_DATA_DIR)

    # -- Load scaling file -----
    YScal = pd.read_csv(
        path + "scaleYc" + nameMech + ".csv",
        header=None, skiprows=1
    ).to_numpy()

    # -- Air / Fuel boundary conditions -----
    p = ct.one_atm
    tin = 298.15

    # -- Build argument list for workers -----
    args_list = [
        (i, phi, p, tin, mech, c, YScal, Soret, CANTERA_DATA_DIR)
        for i, phi in enumerate(phiVector)
    ]

    # -- Run flamelets in parallel -----
    print(f"\nRunning {nPointsZ} flamelets on {N_WORKERS} workers...\n")

    counter = Value(ctypes.c_int, 0)
    with Pool(processes=N_WORKERS,
              initializer=_init_counter,
              initargs=(counter, nPointsZ)) as pool:
        rawResults = pool.map(runFlamelet, args_list)

    # Sort by original phi index to preserve order
    rawResults.sort(key=lambda x: x[0])

    # Rebuild results dict and scalar vectors
    results = {}
    ZVec = np.zeros(nPointsZ)
    sLVec = np.zeros(nPointsZ)
    tauVec = np.zeros(nPointsZ)
    lFVec = np.zeros(nPointsZ)

    for i, phi, data in rawResults:
        results[phi] = data
        ZVec[i] = data["ZPhi"]
        sLVec[i] = data["sL"]
        tauVec[i] = data["tau"]
        lFVec[i] = data["lF"]

    print("\nAll flamelets done. Building tables...")

```

The flamelets are computed in parallel using the `runFlamelet` function. All results corresponding to different ϕ values are stored in the dictionary `results`. Optionally, to enable the use of a different subgrid-scale model for variances in LES, the laminar flame speed `sL`, laminar flame thickness `lF`, and heat release

factor `tau` can also be tabulated.

Once the flamelets are computed, they are assembled into matrices using the `createTables` function, where C varies along the rows and Z along the columns.

```
# -- Convert to DataFrames and optionally save -----
def to_df(arr):
    return pd.DataFrame(arr, columns=phiVector)

CTable_df      = to_df(CTable)
ZTable_df      = to_df(ZTable)
TTable_df      = to_df(TTable)
DTable_df      = to_df(DTable)
rhoTable_df    = to_df(rhoTable)
dHTable_df     = to_df(dHTable)
hTable_df      = to_df(hTable)
kTable_df      = to_df(kTable)
CpTable_df     = to_df(CpTable)
HRRTable_df    = to_df(HRRTable)
MMWTable_df    = to_df(MMWTable)
sourceYcVTable_df = to_df(sourceYcVTable)

sLVec_df = pd.DataFrame(sLVec, index=phiVector, columns=['sL'])
tauVec_df = pd.DataFrame(tauVec, index=phiVector, columns=['tau'])
lFVec_df = pd.DataFrame(lFVec, index=phiVector, columns=['lF'])

omegakTable_df = {sp: to_df(omegakTable[sp]) for sp in species}
YkTable_df     = {sp: to_df(YkTable[sp])     for sp in species}
hkTable_df     = {sp: to_df(hkTable[sp])     for sp in species}
XkTable_df     = {sp: to_df(XkTable[sp])     for sp in species}
DkTable_df     = {sp: to_df(DkTable[sp])     for sp in species}
DkSoretTable_df = {sp: to_df(DkSoretTable[sp]) for sp in species}

if save:
    CTable_df.to_csv(path + "CTable.csv",          index=False)
    sLVec_df.to_csv( path + "sLVec.csv",          index=False)
    lFVec_df.to_csv( path + "lFVec.csv",          index=False)
    tauVec_df.to_csv(path + "tauVec.csv",          index=False)
    ZTable_df.to_csv(path + "ZTable.csv",          index=False)
    TTable_df.to_csv(path + "TTable.csv",          index=False)
    DTable_df.to_csv(path + "DTable.csv",          index=False)
    rhoTable_df.to_csv(path + "rhoTable.csv",       index=False)
    dHTable_df.to_csv(path + "dHTable.csv",       index=False)
    hTable_df.to_csv(path + "hTable.csv",          index=False)
    kTable_df.to_csv(path + "kTable.csv",          index=False)
    CpTable_df.to_csv(path + "CpTable.csv",        index=False)
    HRRTable_df.to_csv(path + "HRRTable.csv",      index=False)
    MMWTable_df.to_csv(path + "MMWTable.csv",      index=False)
    sourceYcVTable_df.to_csv(path + "sourceYcVTable.csv", index=False)
    for sp in species:
        omegakTable_df[sp].to_csv( path + f"omega_{sp}Table.csv", index=False)
        DkTable_df[sp].to_csv(      path + f"D_{sp}Table.csv",      index=False)
        DkSoretTable_df[sp].to_csv(path + f"DSoret_{sp}Table.csv", index=False)
        YkTable_df[sp].to_csv(      path + f"Y_{sp}Table.csv",      index=False)
```

```

        #hkTable_df[sp].to_csv(    path + f"h_{sp}Table.csv",    index=False)
        #XkTable_df[sp].to_csv(    path + f"X_{sp}Table.csv",    index=False)
    print("Tables successfully saved in folder ---> " + nameMech + "!")

    # -- Back to numpy for post-processing -----
    CTable      = CTable_df.to_numpy()
    ZTable      = ZTable_df.to_numpy()
    TTable      = TTable_df.to_numpy()
    DTable      = DTable_df.to_numpy()
    hTable      = hTable_df.to_numpy()
    tauVec      = tauVec_df.to_numpy()
    sLVec       = sLVec_df.to_numpy()
    lFVec       = lFVec_df.to_numpy()
    rhoTable    = rhoTable_df.to_numpy()
    dHTable     = dHTable_df.to_numpy()
    kTable      = kTable_df.to_numpy()
    CpTable     = CpTable_df.to_numpy()
    HRRTable    = HRRTable_df.to_numpy()
    MMWTable    = MMWTable_df.to_numpy()
    sourceYcVTable = sourceYcVTable_df.to_numpy()
    for sp in species:
        omegakTable[sp] = omegakTable_df[sp].to_numpy()
        YkTable[sp]     = YkTable_df[sp].to_numpy()
        hkTable[sp]     = hkTable_df[sp].to_numpy()
        XkTable[sp]     = XkTable_df[sp].to_numpy()
        DkTable[sp]     = DkTable_df[sp].to_numpy()
        DkSoretTable[sp] = DkSoretTable_df[sp].to_numpy()

```

To make them easily readable, the tables are saved in csv format. For this purpose, they are first converted into a `DataFrame`, and then written to file if the flag `save = True` is enabled. Subsequently, they are converted into `numpy` arrays for further processing.

The results obtained from Cantera and interpolated onto the C vector are then ready to be interpolated over the Z vector, which is constant along each row of the j -th column. Since, for very lean flames, the mixture fraction may exceed the range covered by the `ZTable`, values outside the tabulated limits are extrapolated using a linear interpolation defined in the `interpExt` function.

```

# -- Create Z/C grids -----
ZTableExt = np.zeros((nPointsC, nPointsZ))
CTableExt = CTable
for i in range(nPointsZ):
    ZTableExt[:, i] = ZTable[0, i]

# -- Interpolate all tables over Z -----
for i in range(nPointsC):
    ZiSRC = ZTable[i, :]
    ZiDST = ZTableExt[i, :]
    TTable[i, :] = m.interpExt(ZiDST, ZiSRC, TTable[i, :])
    CpTable[i, :] = m.interpExt(ZiDST, ZiSRC, CpTable[i, :])
    DTable[i, :] = m.interpExt(ZiDST, ZiSRC, DTable[i, :])
    kTable[i, :] = m.interpExt(ZiDST, ZiSRC, kTable[i, :])
    rhoTable[i, :] = m.interpExt(ZiDST, ZiSRC, rhoTable[i, :])

```

```

dHTable[i, :] = m.interpExt(ZiDST, ZiSRC, dHTable[i, :])
hTable[i, :] = m.interpExt(ZiDST, ZiSRC, hTable[i, :])
MMWTable[i, :] = m.interpExt(ZiDST, ZiSRC, MMWTable[i, :])
HRRTable[i, :] = m.interpExt(ZiDST, ZiSRC, HRRTable[i, :])
sourceYcVTable[i, :] = m.interpExt(ZiDST, ZiSRC, sourceYcVTable[i, :])
for sp in species:
    YkTable[sp][i, :] = m.interpExt(ZiDST, ZiSRC, YkTable[sp][i, :])
    hkTable[sp][i, :] = m.interpExt(ZiDST, ZiSRC, hkTable[sp][i, :])
    XkTable[sp][i, :] = m.interpExt(ZiDST, ZiSRC, XkTable[sp][i, :])
    DkTable[sp][i, :] = m.interpExt(ZiDST, ZiSRC, DkTable[sp][i, :])
    DkSoretTable[sp][i, :] = m.interpExt(ZiDST, ZiSRC, DkSoretTable[sp][i, :])
    omegakTable[sp][i, :] = m.interpExt(ZiDST, ZiSRC, omegakTable[sp][i, :])

```

Here we arrive at the computation of the correction terms that must be included in the diffusive fluxes of the mixture fraction and progress variable transport equations to account for preferential diffusion effects associated with hydrogen.

```

# -- Gamma coefficients
↪ -----

windowC = 40
windowZ = 40
order = 1

# Gamma_Yc-Yc
gammaYcYc1 = np.zeros((nPointsC, nPointsZ))
LYcC = m.calculateLambdaYkC(DkTable, YkTable, MMWTable, CTableExt, species, 'H2O',
↪ windowC, order)
for i in range(nPointsZ):
    gammaYcYc1[:, i] = LYcC[:, i] / YkTable['H2O'][:, i][-1] - DTable[:, i]
gammaYcYc = (gammaYcYc1 + DTable) * rhoTable

# Gamma_Yc-Z
gammaYcZ = np.zeros((nPointsC, nPointsZ))
dH2OdZ = m.SavitzkyGolay(YkTable['H2O'], ZTableExt, 1, windowC, order)
LYcZ = m.calculateLambdaYkZ(DkTable, YkTable, MMWTable, ZTableExt, species, 'H2O',
↪ windowC, order)
for i in range(nPointsC):
    dcdz_YcConst = -1 / YkTable['H2O'][-1, :] * (CTableExt[i, :] * dH2OdZ[-1, :])
    gammaYcZ[i, :] = LYcC[i, :] * dcdz_YcConst + LYcZ[i, :]
gammaYcZ = gammaYcZ * rhoTable

# Gamma_Z-Yc
gammaZYc = np.zeros((nPointsC, nPointsZ))
LambdakTable = {sp: m.calculateLambdaYkC(DkTable, YkTable, MMWTable, CTableExt, species,
↪ sp, windowZ, order) for sp in species}
gas = ct.Solution(mech)

listSpecies = ['H', 'O']
LZpCTable = {}
for k in listSpecies:
    isp = gas.element_index(k)

```

```

S = sum(
    gas.n_atoms(gas.species_index(sp), isp) /
    ↪ gas.molecular_weights[gas.species_index(sp)] * LambdakTable[sp]
    for sp in species
)
LZpCTable[k] = gas.molecular_weights[gas.species_index(k)] * S

KZ = 1 / (gas.molecular_weights[gas.species_index('H')] * 2) + Y02Air /
    ↪ gas.molecular_weights[gas.species_index('O')]
LZC = 1 / KZ * (LZpCTable['H'] / (2 * gas.molecular_weights[gas.species_index('H')]) -
    ↪ LZpCTable['O'] / gas.molecular_weights[gas.species_index('O')])
for i in range(nPointsZ):
    gammaZYc[:, i] = LZC[:, i] / YkTable['H2O'][:, i][-1]
gammaZYc = gammaZYc * rhoTable

# Gamma_Z-Z
gammaZZ = np.zeros((nPointsC, nPointsZ))
LambdakTable = {sp: m.calculateLambdaYkZ(DkTable, YkTable, MMWTable, ZTableExt, species,
    ↪ sp, windowZ, order) for sp in species}
LZpZTable = {}
for k in listSpecies:
    isp = gas.element_index(k)
    S = sum(
        gas.n_atoms(gas.species_index(sp), isp) /
        ↪ gas.molecular_weights[gas.species_index(sp)] * LambdakTable[sp]
        for sp in species
    )
    LZpZTable[k] = gas.molecular_weights[gas.species_index(k)] * S

LZZ = 1 / KZ * (LZpZTable['H'] / (2 * gas.molecular_weights[gas.species_index('H')]) -
    ↪ LZpZTable['O'] / gas.molecular_weights[gas.species_index('O')])
for i in range(nPointsC):
    dcdz_YcConst = -1 / YkTable['H2O'][-1, :] * (CTableExt[i, :] * dH2OdZ[-1, :])
    gammaZZ[i, :] = LZC[i, :] * dcdz_YcConst + LZZ[i, :] - DTable[i, :]
gammaZZ = (gammaZZ + DTable) * rhoTable

```

A detailed description of the computation of these terms can be found in Pérez et al. [2] and Fortes et al. [1].

```

# --3-D / Contour plots -----
if plotTables:
    m.plot3DTable(CTableExt, ZTableExt, omegakTable['H2O'], "[kg/(m3 \cdot s)]",
        ↪ "\\dot{\\omega}_{\\mathrm{H_2O}}", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, HRRTable, "W/m3",
        ↪ "HRR", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, TTable, "[K]", "T",
        ↪ "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, DTable, "[kg/(m \cdot s)]",
        ↪ "D_{th}", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, DkSoretTable['H2O'], "[m2/s]",
        ↪ "DSoret_{H_2O}", "C", "Z")

```

```

m.plot3DTable(CTableExt, ZTableExt, DkTable['H2O'],      "[m^2/s]",
  ↳ "D_{H_2O}",      "C", "Z")
m.plot3DTable(CTableExt, ZTableExt, CpTable,              "[J/(kg \cdot K)]",
  ↳ "Cp",            "C", "Z")
m.plot3DTable(CTableExt, ZTableExt, kTable,               "[W/(m \cdot K)]",
  ↳ "\\lambda",      "C", "Z")
m.plot3DTable(CTableExt, ZTableExt, dHTable,              "[J/kg]",
  ↳ "\\Delta h_f^o",  "C", "Z")
m.plot3DTable(CTableExt, ZTableExt, MMWTable,            "[kg/mol]",
  ↳ "W_{mix}",       "C", "Z")
m.plot3DTable(CTableExt, ZTableExt, YkTable['H2O'],      "[-]",
  ↳ "Y_{H2O}",       "C", "Z")
m.plot3DTable(CTableExt, ZTableExt, YkTable['OH'],       "[-]",
  ↳ "Y_{OH*}",       "C", "Z")
m.plot3DTable(ZTableExt, CTableExt, gammaYcYc,           "[m^2/s]", "\\rho \cdot
  ↳ "\\Gamma'_{Y_c,Y_c}",      "Z", "C")
m.plot3DTable(ZTableExt, CTableExt, gammaYcZ,            "[m^2/s]", "\\rho \cdot
  ↳ "\\Gamma'_{Y_c,Z}",        "Z", "C")
m.plot3DTable(ZTableExt, CTableExt, gammaZYc,            "[m^2/s]", "\\rho \cdot
  ↳ "\\Gamma'_{Z,Y_c}",        "Z", "C")
m.plot3DTable(ZTableExt, CTableExt, gammaZZ,             "[m^2/s]", "\\rho \cdot
  ↳ "\\Gamma_{Z,Z}",          "Z", "C")

m.plotContourTable(CTableExt, ZTableExt, HRRTable,       "[W/m^3]",
  ↳ "HRR",            "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, omegakTable['H2O'], "[kg/(m^3 \cdot
  ↳ s)]", "\\dot{\\omega}_{\\mathrm{H_2O}}", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, TTable,          "[K]",
  ↳ "T",              "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, DTable,          "[kg/(m \cdot s)]",
  ↳ "D_{th}",         "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, DkSoretTable['H2O'], "[m^2/s]",
  ↳ "DSoret_{H_2O}",  "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, DkTable['H2O'],   "[m^2/s]",
  ↳ "D_{H_2O}",       "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, CpTable,          "[J/(kg \cdot K)]",
  ↳ "Cp",            "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, dHTable,          "[J/kg]",
  ↳ "\\Delta h_f^o",  "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, kTable,           "[W/(m \cdot K)]",
  ↳ "\\lambda",      "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, MMWTable,         "[kg/mol]",
  ↳ "W_{mix}",       "C", "Z")
m.plotContourTable(ZTableExt, CTableExt, gammaYcYc,        "[m^2/s]", "\\rho
  ↳ \cdot \\Gamma_{Y_c,Y_c}",      "Z", "C")
m.plotContourTable(ZTableExt, CTableExt, gammaYcZ,         "[m^2/s]", "\\rho
  ↳ \cdot \\Gamma_{Y_c,Z}",        "Z", "C")
m.plotContourTable(ZTableExt, CTableExt, gammaZYc,         "[m^2/s]", "\\rho
  ↳ \cdot \\Gamma'_{Z,Y_c}",       "Z", "C")
m.plotContourTable(ZTableExt, CTableExt, gammaZZ,          "[m^2/s]", "\\rho
  ↳ \cdot \\Gamma_{Z,Z}",          "Z", "C")

```

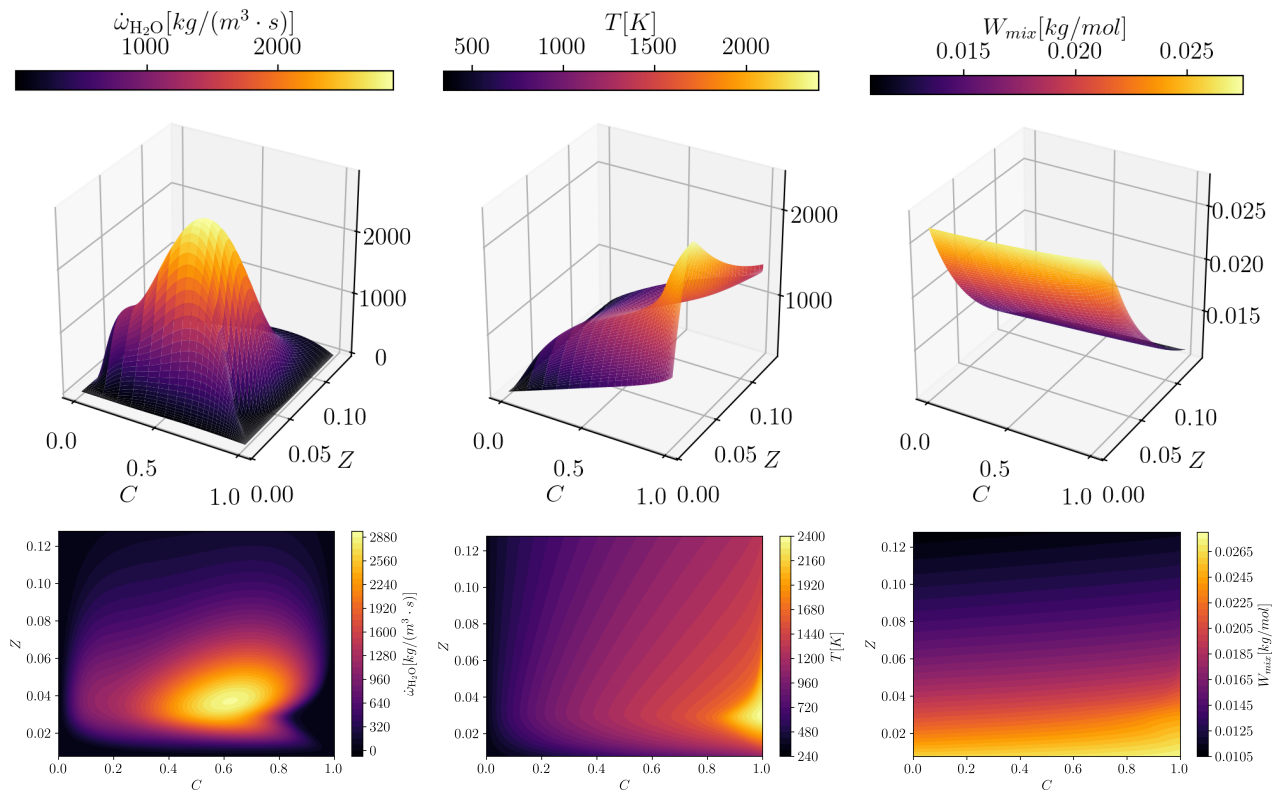


Figure 2. From left to right surface and contour plots in $C - Z$ space of: water vapor reaction rate, temperature and mean molecular weight.

```
print("\nDone.")
```

All plotting functions can be used to generate either surface plots or contour plots for visualizing the flamelets.

2.3. extendFGMTables.py

In this part, the objective is to extend the tables beyond the flammability limits, identified by [zLean](#) and [zRich](#). This is necessary because the integration with the β -PDF is performed over the full interval [0,1]. Moreover, this script allows the flamelets computed in the previous step to be loaded and post-processed if needed, without rerunning all the simulations. The initial part of the code is very similar to the previous script and is therefore omitted here for brevity.

```
# Load Tables
DkTable = {}
DkSoretTable = {}
YkTable = {}
omegakTable = {}
XkTable = {}
hkTable = {}

CTable = pd.read_csv(path + "CTable.csv",header=None, skiprows=1).to_numpy()
ZTable = pd.read_csv(path + "ZTable.csv",header=None, skiprows=1).to_numpy()
TTable = pd.read_csv(path + "TTable.csv",header=None, skiprows=1).to_numpy()
sLVec = pd.read_csv(path + "sLVec.csv",header=None, skiprows=1).to_numpy()
tauVec = pd.read_csv(path + "tauVec.csv",header=None, skiprows=1).to_numpy()
lFVec = pd.read_csv(path + "lFVec.csv",header=None, skiprows=1).to_numpy()
dHTable = pd.read_csv(path + "dHTable.csv",header=None, skiprows=1).to_numpy()
DTable = pd.read_csv(path + "DTable.csv",header=None, skiprows=1).to_numpy()
rhoTable = pd.read_csv(path + "rhoTable.csv",header=None, skiprows=1).to_numpy()
kTable = pd.read_csv(path + "kTable.csv",header=None, skiprows=1).to_numpy()
CpTable = pd.read_csv(path + "CpTable.csv",header=None, skiprows=1).to_numpy()
MMWTable = pd.read_csv(path + "MMWTable.csv",header=None, skiprows=1).to_numpy()
HRRTable = pd.read_csv(path + "HRRTable.csv",header=None, skiprows=1).to_numpy()
hTable = pd.read_csv(path + "hTable.csv",header=None, skiprows=1).to_numpy()
sourceYcVTable = pd.read_csv(path + "sourceYcVTable.csv",header=None,
    ↪ skiprows=1).to_numpy()

for sp in species:
    omegakTable[sp] = pd.read_csv(path + f"omega_{sp}Table.csv",header=None,
    ↪ skiprows=1).to_numpy()
    DkTable[sp] = pd.read_csv(path + f"D_{sp}Table.csv",header=None, skiprows=1).to_numpy()
    DkSoretTable[sp] = pd.read_csv(path + f"DSoret_{sp}Table.csv",header=None,
    ↪ skiprows=1).to_numpy()
    YkTable[sp] = pd.read_csv(path + f"Y_{sp}Table.csv",header=None, skiprows=1).to_numpy()
    XkTable[sp] = pd.read_csv(path + f"X_{sp}Table.csv",header=None, skiprows=1).to_numpy()
    hkTable[sp] = pd.read_csv(path + f"h_{sp}Table.csv",header=None, skiprows=1).to_numpy()

ZVec = ZTable[0,:]
CVec = CTable[:,0]

zLean = ZVec[0]
zRich = ZVec[-1]
# Discretize Mixture Fraction and Progress Variable
nPointsC = len(CVec)
nPointsZ = len(ZVec)
```

```

ZVec = np.zeros(len(ZVec))
YcMax = np.zeros(len(ZVec))
YcMin = np.zeros(len(ZVec))

# Set Thermodynamic parameters for the flamelets
p = ct.one_atm #pressure
tin = 298.15 # unburt gas temperature

# Create Air and Fuel properties
# Air
gasAir = ct.Solution(mech)
air = "O2:0.21,N2:0.79"
gasAir.set_equivalence_ratio(phi=0, fuel="H2:1", oxidizer=air)
gasAir.TPX = tin, p, gasAir.X
YO2Air = gasAir.Y[gasAir.species_index('O2')]

Air = {
    "CpAir"      : gasAir.cp_mass,
    "WAir"       : gasAir.mean_molecular_weight*1e-3,
    "DAir"       : gasAir.thermal_conductivity/(gasAir.density*gasAir.cp_mass),
    "rhoAir"     : gasAir.density,
    "kAir"       : gasAir.thermal_conductivity,
    "HRRAir"     : 0
}

for i, sp in enumerate(species):
    Air[f"D_{sp}Air"] = gasAir.mix_diff_coeffs[gasAir.species_index(sp)]
    Air[f"Y_{sp}Air"] = gasAir.Y[gasAir.species_index(sp)]
    Air[f"X_{sp}Air"] = gasAir.X[gasAir.species_index(sp)]

# Fuel
gasFuel = ct.Solution(mech)
gasFuel.TPX = tin, p, "H2:1"

Fuel = {
    "CpFuel"     : gasFuel.cp_mass,
    "WFuel"      : gasFuel.mean_molecular_weight*1e-3,
    "DFuel"      : gasFuel.thermal_conductivity/(gasFuel.density*gasFuel.cp_mass),
    "rhoFuel"    : gasFuel.density,
    "kFuel"      : gasFuel.thermal_conductivity,
    "HRRFuel"    : 0
}

for i, sp in enumerate(species):
    Fuel[f"D_{sp}Fuel"] = gasFuel.mix_diff_coeffs[gasFuel.species_index(sp)]
    Fuel[f"Y_{sp}Fuel"] = gasFuel.Y[gasFuel.species_index(sp)]
    Fuel[f"X_{sp}Fuel"] = gasFuel.X[gasFuel.species_index(sp)]

```

After loading the tables, the properties of each variable in the pure air and pure fuel streams are evaluated and stored in the dictionaries `Air` and `Fuel`.

Since the tables generated in the previous stage were only interpolated along C , they must now also be interpolated along Z , following the same procedure described earlier. As previously done, the coefficients required to correct the progress variable and mixture fraction transport equations are then computed.

```
# Create Z matrix where Z is constant along the rows and varies along the columns
ZTableExt1 = np.zeros((nPointsC,nPointsZ))
CTableExt = CTable

for i in range(nPointsZ):
    ZTableExt1[:,i] = np.ones(nPointsC)*ZTable[0,i]

# Interpolate over Z
for i in range(nPointsC):
    TTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], TTable[i,:])
    CpTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], CpTable[i,:])
    DTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], DTable[i,:])
    kTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], kTable[i,:])
    rhoTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], rhoTable[i,:])
    dHTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], dHTable[i,:])
    hTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], hTable[i,:])
    MMWTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], MMWTable[i,:])
    HRRTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], HRRTable[i,:])
    sourceYcVTable[i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], sourceYcVTable[i,:])

    for sp in species:
        YkTable[sp][i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], YkTable[sp][i,:])
        hkTable[sp][i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], hkTable[sp][i,:])
        XkTable[sp][i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], XkTable[sp][i,:])
        DkTable[sp][i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:], DkTable[sp][i,:])
        DkSoretTable[sp][i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:],
        ↪ DkSoretTable[sp][i,:])
        omegakTable[sp][i,:] = m.interpExt(ZTableExt1[i,:], ZTable[i,:],
        ↪ omegakTable[sp][i,:])

windowC = 40
windowZ = 40
order = 1

# Gamma_YcYc
gammaYcYc1 = np.zeros((nPointsC, nPointsZ))

if Soret:
    LYcC = m.calculateLambdaYkC(DkTable, DkSoretTable, TTable, rhoTable, YkTable, MMWTable,
    ↪ CTableExt, species, 'H2O', windowC, order)
else:
    LYcC = m.calculateLambdaYkC(DkTable, YkTable, MMWTable, CTableExt, species, 'H2O',
    ↪ windowC, order)
```

```

for i in range(nPointsZ):
    gammaYcYc1[:, i] = LYcC[:, i] / YkTable['H2O'][:, i][-1] - DTable[:, i]
gammaYcYc = (gammaYcYc1 + DTable) * rhoTable

# Gamma_YcZ
gammaYcZ = np.zeros((nPointsC, nPointsZ))
dH2OdZ = m.SavitzkyGolay(YkTable['H2O'], ZTableExt1, 1, windowC, order)

if Soret:
    LYcZ = m.calculateLambdaYkZ(DkTable, DkSoretTable, TTable, rhoTable, YkTable,
    ↪ MMWTable, ZTableExt1, species, 'H2O', windowC, order)
else:
    LYcZ = m.calculateLambdaYkZ(DkTable, YkTable, MMWTable, ZTableExt1, species, 'H2O',
    ↪ windowC, order)

for i in range(nPointsC):
    dcdz_YcConst = -1 / YkTable['H2O'][-1, :] * (CTableExt[i, :] * dH2OdZ[-1, :])
    gammaYcZ[i, :] = LYcC[i, :] * dcdz_YcConst + LYcZ[i, :]
gammaYcZ = gammaYcZ * rhoTable

# Gamma_ZYc
gammaZYc = np.zeros((nPointsC, nPointsZ))

if Soret:
    LambdakTable = {sp: m.calculateLambdaYkC(DkTable, DkSoretTable, TTable, rhoTable,
    ↪ YkTable, MMWTable, CTableExt, species, sp, windowZ, order) for sp in species}
else:
    LambdakTable = {sp: m.calculateLambdaYkC(DkTable, YkTable, MMWTable, CTableExt, species,
    ↪ sp, windowZ, order) for sp in species}

gas = ct.Solution(mech)

listSpecies = ['H', 'O']
LZpCTable = {}
for k in listSpecies:
    isp = gas.element_index(k)
    S = sum(
        gas.n_atoms(gas.species_index(sp), isp) /
        ↪ gas.molecular_weights[gas.species_index(sp)] * LambdakTable[sp]
        for sp in species
    )
    LZpCTable[k] = gas.molecular_weights[gas.species_index(k)] * S

KZ = 1 / (gas.molecular_weights[gas.species_index('H')] * 2) + Y02Air /
    ↪ gas.molecular_weights[gas.species_index('O')]
LZC = 1 / KZ * (LZpCTable['H'] / (2 * gas.molecular_weights[gas.species_index('H')]) -
    ↪ LZpCTable['O'] / gas.molecular_weights[gas.species_index('O')])
for i in range(nPointsZ):
    gammaZYc[:, i] = LZC[:, i] / YkTable['H2O'][:, i][-1]
gammaZYc = gammaZYc * rhoTable

# Gamma_ZZ

```

```

gammaZZ      = np.zeros((nPointsC, nPointsZ))

if Soret:
    LambdakTable = {sp: m.calculateLambdaYkZ(DkTable, DkSoretTable, TTable, rhoTable,
        ↪ YkTable, MMWTable, ZTableExt1, species, sp, windowZ, order) for sp in species}
else:
    LambdakTable = {sp: m.calculateLambdaYkZ(DkTable, YkTable, MMWTable, ZTableExt1,
        ↪ species, sp, windowZ, order) for sp in species}

LZpZTable = {}
for k in listSpecies:
    isp = gas.element_index(k)
    S = sum(
        gas.n_atoms(gas.species_index(sp), isp) /
        ↪ gas.molecular_weights[gas.species_index(sp)] * LambdakTable[sp]
        for sp in species
    )
    LZpZTable[k] = gas.molecular_weights[gas.species_index(k)] * S

LZZ = 1 / KZ * (LZpZTable['H'] / (2 * gas.molecular_weights[gas.species_index('H')]) -
    ↪ LZpZTable['O'] / gas.molecular_weights[gas.species_index('O')])
for i in range(nPointsC):
    dcdz_YcConst = -1 / YkTable['H2O'][-1, :] * (CTableExt[i, :] * dH2OdZ[-1, :])
    gammaZZ[i, :] = LZC[i, :] * dcdz_YcConst + LZZ[i, :] - DTable[i, :]
gammaZZ = (gammaZZ + DTable) * rhoTable

```

The next step is to extend the values beyond the flammability limits. To this end, the mixture fraction vector is divided into three intervals, namely `ZLean`, `ZFlam`, and `ZRich`, each defined using a linear spacing. The complete `Z` vector is then obtained by concatenating these three segments.

Since the data are handled in matrix form, the extended mixture fraction matrix `ZTableExt` and the scaled progress variable matrix `CTableExt` are constructed at this stage.

```

# Extend Tables out of FL
ZLean = np.linspace(0, zLean-1e-4, 50)
ZFlam = np.linspace(ZVec[0], ZVec[-1], 200)
ZRich = np.linspace(zRich+1e-4, 1, 50)

Z = np.concatenate((ZLean, ZFlam, ZRich))
ZTableExt = np.transpose(np.tile(Z[:, None], (1, nPointsC)))
CTableExt = np.tile(CVec[:, None], (1, np.size(Z)))

```

First, scalar quantities such as the laminar flame speed, laminar flame thickness, and heat release factor are interpolated to obtain their values outside the flammability limits.

```

fsLVec = interp1d(np.concatenate(([0], ZVec, [1])), np.concatenate(([0], sLVec.flatten(), [0])),
    ↪ fill_value='extrapolate')
fLFVec = interp1d(np.concatenate(([0], ZVec, [1])), np.concatenate(([0], lFVec.flatten(), [0])),
    ↪ fill_value='extrapolate')
ftauVec = interp1d(np.concatenate(([0], ZVec, [1])),
    ↪ np.concatenate(([0], tauVec.flatten(), [0])), fill_value='extrapolate')

```

```
sLVec = fsLVec(Z)
lFVec = flFVec(Z)
tauVec = ftauVec(Z)
```

Subsequently, the table values are extrapolated beyond the flammability limits using the function `interpExt2`, defined in `modules/functions`.

```
TTableExt = np.zeros((nPointsC, len(Z)))
CpTableExt = np.zeros((nPointsC, len(Z)))
DTableExt = np.zeros((nPointsC, len(Z)))
dHTableExt = np.zeros((nPointsC, len(Z)))
kTableExt = np.zeros((nPointsC, len(Z)))
rhoTableExt = np.zeros((nPointsC, len(Z)))
MMWTableExt = np.zeros((nPointsC, len(Z)))
HRRTableExt = np.zeros((nPointsC, len(Z)))
sourceYcVTableExt = np.zeros((nPointsC, len(Z)))
gammaYcYcTableExt = np.zeros((nPointsC, len(Z)))
gammaYcZTableExt = np.zeros((nPointsC, len(Z)))
gammaZYcTableExt = np.zeros((nPointsC, len(Z)))
gammaZZTableExt = np.zeros((nPointsC, len(Z)))

YkTableExt = {}
XkTableExt = {}
DkTableExt = {}
DkSoretTableExt = {}
omegakTableExt = {}
for sp in species:
    YkTableExt[sp] = np.zeros((nPointsC, len(Z)))
    XkTableExt[sp] = np.zeros((nPointsC, len(Z)))
    DkTableExt[sp] = np.zeros((nPointsC, len(Z)))
    DkSoretTableExt[sp] = np.zeros((nPointsC, len(Z)))
    omegakTableExt[sp] = np.zeros((nPointsC, len(Z)))

# Interpolate over Z
for i in range(nPointsC):
    TTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], TTable[i,:], tin, tin)
    sourceYcVTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:],
        ↪ sourceYcVTable[i,:], 0, 0)
    CpTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], CpTable[i,:],
        ↪ Air["CpAir"], Fuel["CpFuel"])
    DTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], DTable[i,:], Air["DAir"],
        ↪ Fuel["DFuel"])
    dHTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], dHTable[i,:], 0, 0)
    kTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], kTable[i,:], Air["kAir"],
        ↪ Fuel["kFuel"])
    rhoTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], rhoTable[i,:],
        ↪ Air["rhoAir"], Fuel["rhoFuel"])
    MMWTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], MMWTable[i,:],
        ↪ Air["WAir"], Fuel["WFuel"])
```

```

HRRTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], HRRTable[i,:], 0, 0)
gammaYcYcTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], gammaYcYc[i,:],
    ↪ 0, 0)
gammaYcZTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], gammaYcZ[i,:], 0,
    ↪ 0)
gammaZYcTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], gammaZYc[i,:], 0,
    ↪ 0)
gammaZZTableExt[i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:], gammaZZ[i,:], 0,
    ↪ 0)

for sp in species:
    YkTableExt[sp][i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:],
        ↪ YkTable[sp][i,:], Air[f"Y_{sp}Air"], Fuel[f"Y_{sp}Fuel"])
    XkTableExt[sp][i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:],
        ↪ XkTable[sp][i,:], Air[f"X_{sp}Air"], Fuel[f"X_{sp}Fuel"])
    DkTableExt[sp][i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:],
        ↪ DkTable[sp][i,:], Air[f"D_{sp}Air"], Fuel[f"D_{sp}Fuel"])
    omegakTableExt[sp][i,:] = m.interpExt2(ZTableExt[i,:], ZTableExt1[i,:],
        ↪ omegakTable[sp][i,:], 0, 0)

```

As done previously, the tables can now be plotted and saved in the `pathOut` folder defined at the beginning of the script.

```

# 3D Plot of tables
if plotTables == True:

    m.plot3DTable(CTableExt, ZTableExt, omegakTableExt['H2O'], "[kg/(m^3 \cdot
        ↪ s)]", "\dot{\omega}_{\mathrm{H_2O}}", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, HRRTableExt, "[W/m^3]", "HRR", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, sourceYcVTableExt, "[kg/(m^3 \cdot
        ↪ s)]", "\dot{\omega}_{Yc,V}", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, TTableExt, "[K]", "T", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, DTableExt, "[kg/(m \cdot s)]", "D_{th}", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, CpTableExt, "[J/(kg \cdot K)]", "Cp", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, kTableExt, "[W/(m \cdot K)]", "\lambda", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, dHTableExt, "[J/kg]", "\Delta h_f^o", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, MMWTableExt, "[kg/mol]", "W_{mix}", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, YkTableExt['H2O'], "[-]", "Y_{H2O}", "C", "Z")
    m.plot3DTable(CTableExt, ZTableExt, YkTableExt['OH'], "[-]", "Y_{OH*}", "C", "Z")
    m.plot3DTable(ZTableExt, CTableExt, gammaYcYcTableExt, "[m^2/s]", "\rho \cdot
        ↪ \Gamma_{Y_c,Y_c}", "Z", "C")
    m.plot3DTable(ZTableExt, CTableExt, gammaYcZTableExt, "[m^2/s]", "\rho \cdot
        ↪ \Gamma_{Y_c,Z}", "Z", "C")
    m.plot3DTable(ZTableExt, CTableExt, gammaZYcTableExt, "[m^2/s]", "\rho \cdot
        ↪ \Gamma'_{Z,Y_c}", "Z", "C")
    m.plot3DTable(ZTableExt, CTableExt, gammaZZTableExt, "[m^2/s]", "\rho \cdot
        ↪ \Gamma_{Z,Z}", "Z", "C")

```

```

# Contour Plot of tables
m.plotContourTable(CTableExt, ZTableExt, HRRTableExt, "[W/m^3]", "HRR", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, omegakTableExt['H2O'], "[kg/(m^3 \cdot s)]", "\dot{\omega}_{\mathrm{H_2O}}", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, sourceYcVTableExt, "[kg/(m^3 \cdot s)]", "\dot{\omega}_{Yc,V}", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, TTableExt, "[K]", "T", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, DTableExt, "[kg/(m \cdot s)]", "D_{th}", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, CpTableExt, "[J/(kg \cdot K)]", "Cp", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, dHTableExt, "[J/kg]", "\Delta h_f^\circ", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, kTableExt, "[W/(m \cdot K)]", "\lambda", "C", "Z")
m.plotContourTable(CTableExt, ZTableExt, MMWTableExt, "[kg/mol]", "W_{mix}", "C", "Z")
m.plotContourTable(ZTableExt, CTableExt, gammaYcYcTableExt, "[m^2/s]", "\rho \cdot \Gamma_{Y_c,Y_c}", "Z", "C")
m.plotContourTable(ZTableExt, CTableExt, gammaYcZTableExt, "[m^2/s]", "\rho \cdot \Gamma_{Y_c,Z}", "Z", "C")
m.plotContourTable(ZTableExt, CTableExt, gammaZYcTableExt, "[m^2/s]", "\rho \cdot \Gamma'_{Z,Y_c}", "Z", "C")
m.plotContourTable(ZTableExt, CTableExt, gammaZZTableExt, "[m^2/s]", "\rho \cdot \Gamma_{Z,Z}", "Z", "C")

# Save Tables
if save == True:

    os.makedirs(pathOut, exist_ok=True)

    gammaYcYc = pd.DataFrame(gammaYcYcTableExt, columns=Z)
    gammaYcZ = pd.DataFrame(gammaYcZTableExt, columns=Z)
    gammaZYc = pd.DataFrame(gammaZYcTableExt, columns=Z)
    gammaZZ = pd.DataFrame(gammaZZTableExt, columns=Z)
    CTable = pd.DataFrame(CTableExt, columns=Z)
    ZTable = pd.DataFrame(ZTableExt, columns=Z)
    sLVec = pd.DataFrame(sLVec, index=Z, columns=['sL'])
    tauVec = pd.DataFrame(tauVec, index=Z, columns=['tau'])
    lFVec = pd.DataFrame(lFVec, index=Z, columns=['lF'])
    TTable = pd.DataFrame(TTableExt, columns=Z)
    sourceYcVTable = pd.DataFrame(sourceYcVTableExt, columns=Z)
    DTable = pd.DataFrame(DTableExt, columns=Z)
    rhoTable = pd.DataFrame(rhoTableExt, columns=Z)
    dHTable = pd.DataFrame(dHTableExt, columns=Z)
    kTable = pd.DataFrame(kTableExt, columns=Z)
    CpTable = pd.DataFrame(CpTableExt, columns=Z)
    HRRTable = pd.DataFrame(HRRTableExt, columns=Z)
    MMWTable = pd.DataFrame(MMWTableExt, columns=Z)
    for i, sp in enumerate(species):
        omegakTable[sp] = pd.DataFrame(omegakTableExt[sp], columns=Z)
        YkTable[sp] = pd.DataFrame(YkTableExt[sp], columns=Z)
        XkTable[sp] = pd.DataFrame(XkTableExt[sp], columns=Z)
        DkTable[sp] = pd.DataFrame(DkTableExt[sp], columns=Z)

```

```

gammaYcYc.to_csv(pathOut + "GYcYcTable.csv", index=False)
gammaYcZ.to_csv(pathOut + "GYcZTable.csv", index=False)
gammaZYc.to_csv(pathOut + "GZYcTable.csv", index=False)
gammaZZ.to_csv(pathOut + "GZZTable.csv", index=False)
CTable.to_csv(pathOut + "CTable.csv", index=False)
sourceYcVTable.to_csv(pathOut + "sourceYcVTable.csv", index=False)
ZTable.to_csv(pathOut + "ZTable.csv", index=False)
sLVec.to_csv(pathOut + "sLVec.csv", index=False)
lFVec.to_csv(pathOut + "lFVec.csv", index=False)
tauVec.to_csv(pathOut + "tauVec.csv", index=False)
TTable.to_csv(pathOut + "TTable.csv", index=False)
dHTable.to_csv(pathOut + "dHTable.csv", index=False)
DTable.to_csv(pathOut + "DTable.csv", index=False)
rhoTable.to_csv(pathOut + "rhoTable.csv", index=False)
kTable.to_csv(pathOut + "kTable.csv", index=False)
CpTable.to_csv(pathOut + "CpTable.csv", index=False)
HRRTable.to_csv(pathOut + "HRRTable.csv", index=False)
MMWTable.to_csv(pathOut + "MMWTable.csv", index=False)
for i, sp in enumerate(species):
    omegakTable[sp].to_csv(pathOut + f"omega_{sp}Table.csv", index=False)
    DkTable[sp].to_csv(pathOut + f"D_{sp}Table.csv", index=False)
    YkTable[sp].to_csv(pathOut + f"Y_{sp}Table.csv", index=False)
    XkTable[sp].to_csv(pathOut + f"X_{sp}Table.csv", index=False)

print("Tables Succesfully saved in folder ---> " + nameMech + "Extended!")

```

2.4. integrateFGMTables.py

This script is designed to perform the β -PDF integration, enabling the use of the model in combination with turbulence closures.

The code begins by defining a set of configuration inputs:

```
# -----
# CONFIGURATION
# -----

save          = True
plotTables    = False
exportTable   = True
NOEqn         = True
Soret         = False

nameMech      = 'SanDiegoNO'
mech          = nameMech + '.yaml'

N_WORKERS     = max(1, os.cpu_count() - 1)

if Soret:
    path = "./" + nameMech + "SoretExtended/"
    YScal = pd.read_csv(nameMech + "Soret/scaleYc" + nameMech + ".csv", header=None,
        ↪ skiprows=1).to_numpy()
    ZTable = pd.read_csv(nameMech + "Soret/ZTable.csv", header=None, skiprows=1).to_numpy()
    print("Soret activated!")
else:
    path = "./" + nameMech + "Extended/"
    YScal = pd.read_csv(nameMech + "/scaleYc" + nameMech + ".csv", header=None,
        ↪ skiprows=1).to_numpy()
    ZTable = pd.read_csv(nameMech + "/ZTable.csv", header=None, skiprows=1).to_numpy()
    print("Soret deactivated!")
```

The number of points used to discretize the controlling variables is then specified. Since the β -PDF integration is performed over the interval $[0, 1]$, the mixture fraction vector is extended by adding 15 points on both the lean and rich sides.

```
# -----
# DISCRETIZATION
# -----

nCTilde = 150
nZTilde = 120
nCVar    = 11
nZVar    = 3

nPointsZLean = 15
nPointsZRich = 15

ZVec = ZTable[0, :]
zLean = ZVec[0]
zRich = ZVec[-1]
```

```

a = 3
s = np.linspace(0, 1, nZTilde)
ZVec = zLean + (zRich - zLean) * (np.exp(a * s) - 1) / (np.exp(a) - 1)

# Z lean / rich extensions
ZLean = np.linspace(0, zLean - 1e-4, nPointsZLean)
ZRich = np.linspace(zRich + 1e-4, 1, nPointsZRich)
ZTilde01 = np.concatenate((ZLean, ZVec, ZRich))
nZTilde = len(ZTilde01)

a = -6
x = np.linspace(0, 1, nCTilde)
CTilde01 = (np.exp(a * x) - 1) / (np.exp(a) - 1)

p = ct.one_atm
tin = 298.15

```

Fuel and air properties are computed as in the previous scripts, after which the tables generated by the script `extendFGMtables.py` over the interval $Z \in [0, 1]$ are loaded.

```

# -----
# Air / Fuel boundary properties
# -----
gas = ct.Solution(mech)
gasAir = ct.Solution(mech)
species = gasAir.species_names
air = "O2:0.21,N2:0.79"
gasAir.set_equivalence_ratio(phi=0, fuel="H2:1", oxidizer=air)
gasAir.TPX = tin, p, gasAir.X
Y02Air = gasAir.Y[gasAir.species_index('O2')]

Air = {
    "CpAir" : gasAir.cp_mass,
    "WAir" : gasAir.mean_molecular_weight * 1e-3,
    "DAir" : gasAir.thermal_conductivity / (gasAir.density * gasAir.cp_mass),
    "rhoAir" : gasAir.density,
    "kAir" : gasAir.thermal_conductivity,
    "HRRAir" : 0,
}
for sp in species:
    Air[f"D_{sp}Air"] = gasAir.mix_diff_coeffs_mass[gasAir.species_index(sp)]
    Air[f"Y_{sp}Air"] = gasAir.Y[gasAir.species_index(sp)]
    Air[f"X_{sp}Air"] = gasAir.X[gasAir.species_index(sp)]

gasFuel = ct.Solution(mech)
gasFuel.TPX = tin, p, "H2:1"
Fuel = {
    "CpFuel" : gasFuel.cp_mass,
    "WFuel" : gasFuel.mean_molecular_weight * 1e-3,
    "DFuel" : gasFuel.thermal_conductivity / (gasFuel.density * gasFuel.cp_mass),

```

```

    "rhoFuel"      : gasFuel.density,
    "kFuel"       : gasFuel.thermal_conductivity,
    "HRRFuel"     : 0,
}
for sp in species:
    Fuel[f"D_{sp}Fuel"] = gasFuel.mix_diff_coeffs_mass[gasFuel.species_index(sp)]
    Fuel[f"Y_{sp}Fuel"] = gasFuel.Y[gasFuel.species_index(sp)]
    Fuel[f"X_{sp}Fuel"] = gasFuel.X[gasFuel.species_index(sp)]

# -----
# LOAD FGM TABLES
# -----
DkTable      = {}
YkTable      = {}
omegakTable  = {}
XkTable      = {}

CTable       = pd.read_csv(path + "CTable.csv",          header=None,
    ↪ skiprows=1).to_numpy()
ZTable       = pd.read_csv(path + "ZTable.csv",          header=None,
    ↪ skiprows=1).to_numpy()
sLVec        = pd.read_csv(path + "sLVec.csv",           header=None,
    ↪ skiprows=1).to_numpy()
tauVec       = pd.read_csv(path + "tauVec.csv",          header=None,
    ↪ skiprows=1).to_numpy()
lFVec        = pd.read_csv(path + "lFVec.csv",           header=None,
    ↪ skiprows=1).to_numpy()
TTable       = pd.read_csv(path + "TTable.csv",          header=None,
    ↪ skiprows=1).to_numpy()
DTable       = pd.read_csv(path + "DTable.csv",          header=None,
    ↪ skiprows=1).to_numpy()
rhoTable     = pd.read_csv(path + "rhoTable.csv",        header=None,
    ↪ skiprows=1).to_numpy()
dHTable      = pd.read_csv(path + "dHTable.csv",         header=None,
    ↪ skiprows=1).to_numpy()
kTable       = pd.read_csv(path + "kTable.csv",          header=None, skiprows=1).to_numpy()
CpTable      = pd.read_csv(path + "CpTable.csv",         header=None,
    ↪ skiprows=1).to_numpy()
MMWTable     = pd.read_csv(path + "MMWTable.csv",        header=None,
    ↪ skiprows=1).to_numpy()
HRRTable     = pd.read_csv(path + "HRRTable.csv",        header=None,
    ↪ skiprows=1).to_numpy()
GYcYcTable   = pd.read_csv(path + "GYcYcTable.csv",      header=None,
    ↪ skiprows=1).to_numpy()
GYcZTable    = pd.read_csv(path + "GYcZTable.csv",      header=None,
    ↪ skiprows=1).to_numpy()
GZYcTable    = pd.read_csv(path + "GZYcTable.csv",      header=None,
    ↪ skiprows=1).to_numpy()
GZZTable     = pd.read_csv(path + "GZZTable.csv",       header=None,
    ↪ skiprows=1).to_numpy()
sourceYcVTable = pd.read_csv(path + "sourceYcVTable.csv", header=None,
    ↪ skiprows=1).to_numpy()

```

```

for sp in species:
    omegakTable[sp] = pd.read_csv(path + f"omega_{sp}Table.csv", header=None,
        ↪ skiprows=1).to_numpy()
    DkTable[sp]      = pd.read_csv(path + f"D_{sp}Table.csv",      header=None,
        ↪ skiprows=1).to_numpy()
    YkTable[sp]      = pd.read_csv(path + f"Y_{sp}Table.csv",      header=None,
        ↪ skiprows=1).to_numpy()

```

The tables are then interpolated onto the new Z and C vectors, denoted as `ZTilde` and `CTilde`:

```

nPointsZ = len(ZTable[0,:])
nPointsC = len(CTable[:,0])

# -----
# STEP 1 - Interpolate tables onto CTilde / ZTilde grids
# -----
# -- interpolate over C axis -----
def _interp_col_C(arr):
    out = np.zeros((nCTilde, nPointsZ))
    for i in range(nPointsZ):
        out[:, i] = m.interpExt(CTilde01, CTable[:, 0], arr[:, i])
    return out

TCTilde      = _interp_col_C(TTable)
WCTilde      = _interp_col_C(MMWTable)
DCTilde      = _interp_col_C(DTable)
kCTilde      = _interp_col_C(kTable)
CpCTilde     = _interp_col_C(CpTable)
dHCTilde     = _interp_col_C(dHTable)
rhoCTilde    = _interp_col_C(rhoTable)
HRRCTilde    = _interp_col_C(HRRTable)
GYcYcCTilde  = _interp_col_C(GYcYcTable)
GYcZCTilde   = _interp_col_C(GYcZTable)
GZYcCTilde   = _interp_col_C(GZYcTable)
GZZCTilde    = _interp_col_C(GZZTable)
sourceYcCTilde = _interp_col_C(omegakTable['H2O'])
sourceH2CTilde = _interp_col_C(omegakTable['H2'])
sourceYcVCTilde = _interp_col_C(sourceYcVTable)
if NOEqn:
    sourceNOCTilde = _interp_col_C(omegakTable['NO'])
YkCTilde = {sp: _interp_col_C(YkTable[sp]) for sp in species}
DkCTilde = {sp: _interp_col_C(DkTable[sp]) for sp in species}

CCTilde = np.tile(CTilde01[:, None], (1, nPointsZ))

# -- interpolate over Z axis -----
def _interp_row_Z(arr_CTilde):
    out = np.zeros((nCTilde, nZTilde))

```

```

for i in range(nCTilde):
    out[i, :] = m.interpExt(ZTilde01, ZTable[0, :], arr_CTilde[i, :])
return out

WZTilde      = _interp_row_Z(WCTilde)
DZTilde      = _interp_row_Z(DCTilde)
dHZTilde     = _interp_row_Z(dHCTilde)
kZTilde      = _interp_row_Z(kCTilde)
CpZTilde     = _interp_row_Z(CpCTilde)
rhoZTilde    = _interp_row_Z(rhoCTilde)
HRRZTilde    = _interp_row_Z(HRRCTilde)
TZTilde      = _interp_row_Z(TCTilde)
GYcYcZTilde  = _interp_row_Z(GYcYcCTilde)
GYcZZTilde   = _interp_row_Z(GYcZCTilde)
GZYcZTilde   = _interp_row_Z(GZYcCTilde)
GZZZTilde    = _interp_row_Z(GZZCTilde)
sourceYcZTilde = _interp_row_Z(sourceYcCTilde)
sourceH2ZTilde = _interp_row_Z(sourceH2CTilde)
sourceYcVZTilde = _interp_row_Z(sourceYcVCTilde)
if NOEqn:
    sourceNOZTilde = _interp_row_Z(sourceNOCTilde)
YkZTilde = {sp: _interp_row_Z(YkCTilde[sp]) for sp in species}
DkZTilde = {sp: _interp_row_Z(DkCTilde[sp]) for sp in species}

CZTilde = np.tile(CTilde01[:, None], (1, nZTilde))

```

The first step is to perform the integration over the C space. To this end, the variance of C must be defined, which is discretized using an exponential distribution. The variance ranges between 0 and 0.25, as 0.25 represents the maximum value attainable by the variance of a β -distribution.

```

# -----
# STEP 2 - Beta parameters for C integration
# -----

iexp      = np.arange(1, nCVar + 1, 1)
CVarMax01 = 0.25 * ((iexp - 1) / (nCVar - 1)) ** 2.5

[CVAR01, CTILDE01] = np.meshgrid(CVarMax01, CTilde01)
CTILDE = CTILDE01[1:-1, 1:-1]
CVAR   = CVAR01[1:-1, 1:-1]
CVAR   = np.minimum(CTILDE * (1 - CTILDE), CVAR)
idx     = CVAR == CTILDE * (1 - CTILDE)
CVAR[idx] = CVAR[idx] - 1e-8

a_C = CTILDE * (CTILDE * (1 - CTILDE) / CVAR - 1)
b_C = a_C * (1 / CTILDE - 1)

nPOINTSC = 1000
C         = np.linspace(0, 1, nPOINTSC)
Z         = ZTilde01
[ZZ, CC] = np.meshgrid(Z, C)

```

The integration is performed using integration by parts, so that the generic variable ζ is integrated as follows:

$$\bar{\zeta} = \int_0^1 \zeta(X) P(X; \tilde{X}, \sigma_x) dX = \zeta(X=1)CDF(X=1) - \int_0^1 \frac{\partial \zeta(X)}{\partial X} CDF(\zeta; \tilde{X}, \sigma_x) dX \quad (3)$$

where CDF represents the cumulative distribution function. The derivative of the generic variable ζ with respect to C must be computed; therefore, the variables are interpolated onto a refined C vector consisting of 1000 points:

```
# -- interpolate tables onto fine C / Z grids -----
def _interp_fine_Z(arr_ZTilde):
    out = np.zeros((nPOINTSC, nZTilde))
    for i in range(nZTilde):
        out[:, i] = m.interpExt(CC[:, 0], CZTilde[:, 0], arr_ZTilde[:, i])
    return out

W_fine      = _interp_fine_Z(WZTilde)
T_fine      = _interp_fine_Z(TZTilde)
dH_fine     = _interp_fine_Z(dHZTilde)
D_fine      = _interp_fine_Z(DZTilde)
k_fine      = _interp_fine_Z(kZTilde)
Cp_fine     = _interp_fine_Z(CpZTilde)
rho_fine    = _interp_fine_Z(rhoZTilde)
HRR_fine    = _interp_fine_Z(HRRZTilde)
GYcYc_fine  = _interp_fine_Z(GYcYcZTilde)
GYcZ_fine   = _interp_fine_Z(GYcZZTilde)
GZYc_fine   = _interp_fine_Z(GZYcZTilde)
GZZ_fine    = _interp_fine_Z(GZZZTilde)
sourceYc_fine = _interp_fine_Z(sourceYcZTilde)
sourceH2_fine = _interp_fine_Z(sourceH2ZTilde)
sourceYcV_fine = _interp_fine_Z(sourceYcVZTilde)
if NOEqn:
    sourceNO_fine = _interp_fine_Z(sourceNOZTilde)
Yk_fine = {sp: _interp_fine_Z(YkZTilde[sp]) for sp in species}
Dk_fine = {sp: _interp_fine_Z(DkZTilde[sp]) for sp in species}

# -- Savitzky-Golay derivatives over C -----
dWdC      = m.SavitzkyGolay(W_fine,      CC, 0, 2, 1)
dTdC      = m.SavitzkyGolay(T_fine,      CC, 0, 2, 1)
ddHdC     = m.SavitzkyGolay(dH_fine,     CC, 0, 2, 1)
dDdC      = m.SavitzkyGolay(D_fine,      CC, 0, 2, 1)
dkdC      = m.SavitzkyGolay(k_fine,      CC, 0, 2, 1)
dCpdC     = m.SavitzkyGolay(Cp_fine,     CC, 0, 2, 1)
drhodC    = m.SavitzkyGolay(rho_fine,    CC, 0, 2, 1)
dHRRdC    = m.SavitzkyGolay(HRR_fine,    CC, 0, 2, 1)
dGYcYcdC  = m.SavitzkyGolay(GYcYc_fine,  CC, 0, 2, 1)
dGYcZdC   = m.SavitzkyGolay(GYcZ_fine,   CC, 0, 2, 1)
dGZYcdC   = m.SavitzkyGolay(GZYc_fine,   CC, 0, 2, 1)
dGZZdC    = m.SavitzkyGolay(GZZ_fine,    CC, 0, 2, 1)
dsourceYcdC = m.SavitzkyGolay(sourceYc_fine, CC, 0, 2, 1)
dsourceYcVdC = m.SavitzkyGolay(sourceYcV_fine, CC, 0, 2, 1)
```

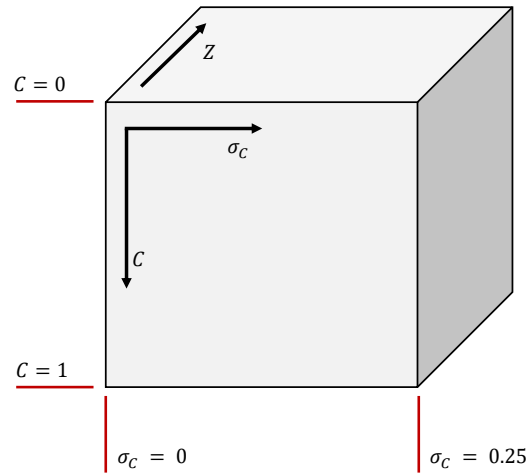


Figure 3. 3-D matrix with extrema at $C = 0$, $C = 1$, $\sigma_C = 0$ and $\sigma_C = 0.25$ excluded.

```

dsourceH2dC = m.SavitzkyGolay(sourceH2_fine, CC, 0, 2, 1)
if NOEqn:
    dsourceNOdC = m.SavitzkyGolay(sourceNO_fine, CC, 0, 2, 1)
dYkdC = {sp: m.SavitzkyGolay(Yk_fine[sp], CC, 0, 2, 1) for sp in species}
dDkdC = {sp: m.SavitzkyGolay(Dk_fine[sp], CC, 0, 2, 1) for sp in species}

```

Since for $C = 0$ and $C = 1$ the β -PDF can exhibit singular (infinite) values, the endpoints are excluded during the integration stage and reintroduced manually afterwards. To provide an overview of the resulting 3D matrix structure, a representation is shown in Fig. 3, where the coordinate vectors are illustrated. To accelerate this stage, the integration is performed in parallel.

```

# -----
# PARALLEL BETA-PDF INTEGRATION OVER C
# -----

def _init_intC(shared_arrays, sp_list, noe, counter, total):
    global _arrC, _species_list, _NOEqn, _counter, _total
    _arrC = shared_arrays
    _species_list = sp_list
    _NOEqn = noe
    _counter = counter
    _total = total

def _worker_intC(args):
    """Compute the Beta-PDF integral over C for one (i, j) point."""
    i, j, a1, b1 = args

```

```

import numpy as np
from scipy.stats import beta as beta_dist

with _counter.get_lock():
    _counter.value += 1
    done = _counter.value
pct = done / _total * 100
filled = int(40 * done / _total)
bar = '' * filled + '' * (40 - filled)
end = '\n' if done == _total else '\r'
print(f" Integration over C {done:>5}/{_total} {pct:5.1f}% [{bar}]", end=end,
      ↪ flush=True)

C_loc = _arrC['C']
CDF = beta_dist.cdf(C_loc, a1, b1)

result = {
    'i': i, 'j': j,
    'W': CDF[-1] * _arrC['W_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dWdC'], C_loc, axis=0),
    'dH': CDF[-1] * _arrC['dH_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['ddHdC'], C_loc, axis=0),
    'D': CDF[-1] * _arrC['D_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dDdC'], C_loc, axis=0),
    'rho': CDF[-1] * _arrC['rho_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['drhodC'], C_loc, axis=0),
    'k': CDF[-1] * _arrC['k_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dkdC'], C_loc, axis=0),
    'Cp': CDF[-1] * _arrC['Cp_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dCpdC'], C_loc, axis=0),
    'HRR': CDF[-1] * 0 - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dHRRdC'], C_loc, axis=0),
    'T': CDF[-1] * _arrC['T_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dTdC'], C_loc, axis=0),
    'GYcYc': CDF[-1] * _arrC['GYcYc_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dGYcYcdC'], C_loc, axis=0),
    'GYcZ': CDF[-1] * _arrC['GYcZ_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dGYcZdC'], C_loc, axis=0),
    'GZYc': CDF[-1] * _arrC['GZYc_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dGZYcdC'], C_loc, axis=0),
    'GZZ': CDF[-1] * _arrC['GZZ_fine'][-1,:] - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dGZZdC'], C_loc, axis=0),
    'sourceH2': CDF[-1] * 0 - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dsourceH2dC'], C_loc, axis=0),
    'sourceYc': CDF[-1] * 0 - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dsourceYcdC'], C_loc, axis=0),
    'sourceYcV': CDF[-1] * 0 - np.trapezoid(CDF[:, None] *
    ↪ _arrC['dsourceYcVdC'], C_loc, axis=0),
}
if _NOEqn:
    result['sourceNO'] = (CDF[-1] * _arrC['sourceNO_fine'][-1,:]
    ↪ - np.trapezoid(CDF[:, None] * _arrC['dsourceNOdC'], C_loc,
    ↪ axis=0))

```

```

result['Yk'] = {sp: CDF[-1] * _arrC['Yk_fine'][sp][-1, :]  
                - np.trapezoid(CDF[:, None] * _arrC['dYkdC'][sp], C_loc, axis=0)  
                for sp in _species_list}  
result['Dk'] = {sp: CDF[-1] * _arrC['Dk_fine'][sp][-1, :]  
                - np.trapezoid(CDF[:, None] * _arrC['dDkdC'][sp], C_loc, axis=0)  
                for sp in _species_list}  
return result  
  
# -----  
# PARALLEL BETA-PDF INTEGRATION OVER Z - top-level worker functions  
# -----  
def _init_intZ(shared_arrays, sp_list, noe, counter, total):  
    global _arrZ, _species_listZ, _NOEqnZ, _counterZ, _totalZ  
    _arrZ = shared_arrays  
    _species_listZ = sp_list  
    _NOEqnZ = noe  
    _counterZ = counter  
    _totalZ = total  
  
def _worker_intZ(args):  
    """Compute the Beta-PDF integral over Z for one (i, j) point."""  
    i, j, a1, b1 = args  
  
    import numpy as np  
    from scipy.stats import beta as beta_dist  
  
    with _counterZ.get_lock():  
        _counterZ.value += 1  
        done = _counterZ.value  
        pct = done / _totalZ * 100  
        filled = int(40 * done / _totalZ)  
        bar = ' ' * filled + '|' * (40 - filled)  
        end = '\n' if done == _totalZ else '\r'  
        print(f" Integration over Z   {done:>5}/{_totalZ}   {pct:5.1f}%   [{bar}]", end=end,  
              flush=True)  
  
        Z01_loc = _arrZ['Z01']  
        CDF = beta_dist.cdf(Z01_loc, a1, b1)  
  
    def _intZ(f_last, df):  
        return CDF[-1] * f_last - np.trapezoid(CDF[:, None, None] * df, Z01_loc, axis=0)  
  
    result = {  
        'i': i, 'j': j,  
        'W': _intZ(_arrZ['W1'][-1], _arrZ['dWdZ']),  
        'dH': _intZ(_arrZ['dH1'][-1], _arrZ['ddHdZ']),  
        'T': _intZ(_arrZ['T1'][-1], _arrZ['dTdZ']),  
        'Cp': _intZ(_arrZ['Cp1'][-1], _arrZ['dCpdZ']),  
        'D': _intZ(_arrZ['D1'][-1], _arrZ['dDdZ']),  
        'k': _intZ(_arrZ['k1'][-1], _arrZ['dkdZ']),

```

```

    'rho':      _intZ(_arrZ['rho1'][-1],      _arrZ['drhodZ']),
    'HRR':      _intZ(_arrZ['HRR1'][-1],      _arrZ['dHRRdZ']),
    'GYcYc':    _intZ(_arrZ['GYcYc1'][-1],    _arrZ['dGYcYcdZ']),
    'GYcZ':     _intZ(_arrZ['GYcZ1'][-1],     _arrZ['dGYcZdZ']),
    'GZYc':     _intZ(_arrZ['GZYc1'][-1],     _arrZ['dGZYcdZ']),
    'GZZ':      _intZ(_arrZ['GZZ1'][-1],      _arrZ['dGZZdZ']),
    'sourceYc': CDF[-1] * 0 - np.trapezoid(CDF[:, None, None] * _arrZ['dsourceYcdZ'],
    ↪ Z01_loc, axis=0),
    'sourceYcV': CDF[-1] * 0 - np.trapezoid(CDF[:, None, None] * _arrZ['dsourceYcVdZ'],
    ↪ Z01_loc, axis=0),
    'sourceH2': CDF[-1] * 0 - np.trapezoid(CDF[:, None, None] * _arrZ['dsourceH2dZ'],
    ↪ Z01_loc, axis=0),
}
if _NOEqnZ:
    result['sourceNO'] = _intZ(_arrZ['sourceNO1'][-1], _arrZ['dsourceNOdZ'])
result['Yk'] = {sp: _intZ(_arrZ['Yk1'][sp][-1], _arrZ['dYkdZ'][sp]) for sp in
    ↪ _species_listZ}
result['Dk'] = {sp: _intZ(_arrZ['Dk1'][sp][-1], _arrZ['dDkdZ'][sp]) for sp in
    ↪ _species_listZ}
return result

if __name__ == "__main__":

    # Pack all arrays needed by C-integration workers into one dict
    shared_arrC = {
        'C': C,
        'W_fine': W_fine, 'dWdC': dWdC,
        'dH_fine': dH_fine, 'ddHdC': ddHdC,
        'D_fine': D_fine, 'dDdC': dDdC,
        'rho_fine': rho_fine, 'drhodC': drhodC,
        'k_fine': k_fine, 'dkdC': dkdC,
        'Cp_fine': Cp_fine, 'dCpdC': dCpdC,
        'HRR_fine': HRR_fine, 'dHRRdC': dHRRdC,
        'T_fine': T_fine, 'dTdC': dTdC,
        'GYcYc_fine': GYcYc_fine, 'dGYcYcdC': dGYcYcdC,
        'GYcZ_fine': GYcZ_fine, 'dGYcZdC': dGYcZdC,
        'GZYc_fine': GZYc_fine, 'dGZYcdC': dGZYcdC,
        'GZZ_fine': GZZ_fine, 'dGZZdC': dGZZdC,
        'sourceYc_fine': sourceYc_fine, 'dsourceYcdC': dsourceYcdC,
        'sourceYcV_fine': sourceYcV_fine, 'dsourceYcVdC': dsourceYcVdC,
        'sourceH2_fine': sourceH2_fine, 'dsourceH2dC': dsourceH2dC,
        'Yk_fine': Yk_fine, 'dYkdC': dYkdC,
        'Dk_fine': Dk_fine, 'dDkdC': dDkdC,
    }
    if NOEqn:
        shared_arrC['sourceNO_fine'] = sourceNO_fine
        shared_arrC['dsourceNOdC'] = dsourceNOdC

    # Build task list: one task per (i, j) pair
    tasks_C = [
        (i, j, float(a_C[i, j]), float(b_C[i, j]))
        for i in range(nCTilde - 2)

```

```

    for j in range(nCVar - 2)
]

print(f"\nBeta-PDF integration over C - {len(tasks_C)} tasks on {N_WORKERS} workers...")

counter_C = Value(ctypes.c_int, 0)

WIntOverC      = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
DIntOverC      = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
dHIntOverC     = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
kIntOverC      = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
CpIntOverC     = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
rhoIntOverC    = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
TIntOverC      = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
HRRIntOverC    = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
GYcYcIntOverC  = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
GYcZIntOverC   = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
GZYcIntOverC   = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
GZZIntOverC    = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
sourceYcIntOverC = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
sourceYcVIntOverC = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
sourceH2IntOverC = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
if NOEqn:
    sourceNOIntOverC = np.zeros((nZTilde, nCTilde - 2, nCVar - 2))
YkIntOverC = {sp: np.zeros((nZTilde, nCTilde - 2, nCVar - 2)) for sp in species}
DkIntOverC = {sp: np.zeros((nZTilde, nCTilde - 2, nCVar - 2)) for sp in species}

resC_list = []
with Pool(processes=N_WORKERS,
          initializer=_init_intC,
          initargs=(shared_arrC, species, NOEqn, counter_C, len(tasks_C))) as pool:
    for res in pool.imap_unordered(_worker_intC, tasks_C):
        resC_list.append(res)

# Unpack results into pre-allocated arrays
for res in resC_list:
    i, j = res['i'], res['j']
    WIntOverC[:, i, j] = res['W']
    dHIntOverC[:, i, j] = res['dH']
    DIntOverC[:, i, j] = res['D']
    rhoIntOverC[:, i, j] = res['rho']
    kIntOverC[:, i, j] = res['k']
    CpIntOverC[:, i, j] = res['Cp']
    HRRIntOverC[:, i, j] = res['HRR']
    TIntOverC[:, i, j] = res['T']
    GYcYcIntOverC[:, i, j] = res['GYcYc']
    GYcZIntOverC[:, i, j] = res['GYcZ']
    GZYcIntOverC[:, i, j] = res['GZYc']
    GZZIntOverC[:, i, j] = res['GZZ']
    sourceYcIntOverC[:, i, j] = res['sourceYc']
    sourceYcVIntOverC[:, i, j] = res['sourceYcV']
    sourceH2IntOverC[:, i, j] = res['sourceH2']
    if NOEqn:

```

```

        sourceNOIntOverC[:, i, j] = res['sourceNO']
    for sp in species:
        YkIntOverC[sp][:, i, j] = res['Yk'][sp]
        DkIntOverC[sp][:, i, j] = res['Dk'][sp]

    print("Integration over C completed!")

```

Once the integration is completed, the boundary values are reintroduced manually using the `addBoundaries3D` function.

```

# -- Add C-boundaries -----
WIntOverC01      = m.addBoundaries3D(WIntOverC,      WZTilde,      CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
dHIntOverC01     = m.addBoundaries3D(dHIntOverC,     dHZTilde,     CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
TIntOverC01      = m.addBoundaries3D(TIntOverC,      TZTilde,      CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
DIntOverC01      = m.addBoundaries3D(DIntOverC,      DZTilde,      CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
rhoIntOverC01    = m.addBoundaries3D(rhoIntOverC,    rhoZTilde,    CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
CpIntOverC01     = m.addBoundaries3D(CpIntOverC,     CpZTilde,     CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
kIntOverC01      = m.addBoundaries3D(kIntOverC,      kZTilde,      CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
HRRIntOverC01    = m.addBoundaries3D(HRRIntOverC,    HRRZTilde,    CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
GYcYcIntOverC01  = m.addBoundaries3D(GYcYcIntOverC,  GYcYcZTilde,  CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
GYcZIntOverC01   = m.addBoundaries3D(GYcZIntOverC,   GYcZZTilde,   CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
GZYcIntOverC01   = m.addBoundaries3D(GZYcIntOverC,   GZYcZTilde,   CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
GZZIntOverC01    = m.addBoundaries3D(GZZIntOverC,    GZZZTilde,    CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
sourceYcIntOverC01 = m.addBoundaries3D(sourceYcIntOverC, sourceYcZTilde, CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
sourceYcVIntOverC01 = m.addBoundaries3D(sourceYcVIntOverC, sourceYcVZTilde, CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
sourceH2IntOverC01 = m.addBoundaries3D(sourceH2IntOverC, sourceH2ZTilde, CZTilde,
    ↪ CTilde01, nZTilde, nCTilde, nCVar)
if NOEqn:
    sourceNOIntOverC01 = m.addBoundaries3D(sourceNOIntOverC, sourceNOZTilde, CZTilde,
        ↪ CTilde01, nZTilde, nCTilde, nCVar)
YkIntOverC01 = {sp: m.addBoundaries3D(YkIntOverC[sp], YkZTilde[sp], CZTilde, CTilde01,
    ↪ nZTilde, nCTilde, nCVar) for sp in species}
DkIntOverC01 = {sp: m.addBoundaries3D(DkIntOverC[sp], DkZTilde[sp], CZTilde, CTilde01,
    ↪ nZTilde, nCTilde, nCVar) for sp in species}

    print("Added boundaries for C = 0, C = 1, sigma_c = 0, sigma_c = sigma_c_MAX!")

```

At this stage, the integration over C is completed and the integration over Z can be performed. The procedure follows the same approach, and the final result is a four-dimensional matrix.

First, the matrices are interpolated onto a refined vector with 1000 points. To achieve this, they are reshaped into a two-dimensional matrix of size `nZTilde x (nCTilde x nCVar)`. After interpolation, the matrices are reshaped back to their original three-dimensional form, as illustrated in Fig. 3.

```
# -----
# STEP 3 - Prepare for Z integration
# -----
nPOINTS_Z = 1000
Z01 = np.linspace(0, 1, nPOINTS_Z)
Z = Z01[1:-1]

WIntOverC01Reshaped = WIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
TIntOverC01Reshaped = TIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
DIntOverC01Reshaped = DIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
dHIntOverC01Reshaped = dHIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
kIntOverC01Reshaped = kIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
CpIntOverC01Reshaped = CpIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
rhoIntOverC01Reshaped = rhoIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
HRRIntOverC01Reshaped = HRRIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
GYcYcIntOverC01Reshaped = GYcYcIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
GYcZIntOverC01Reshaped = GYcZIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
GZYcIntOverC01Reshaped = GZYcIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
GZZIntOverC01Reshaped = GZZIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
sourceYcIntOverC01Reshaped = sourceYcIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
sourceYcVIntOverC01Reshaped = sourceYcVIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
sourceH2IntOverC01Reshaped = sourceH2IntOverC01.reshape(nZTilde, nCTilde * nCVar).T
if NOEqn:
    sourceNOIntOverC01Reshaped = sourceNOIntOverC01.reshape(nZTilde, nCTilde * nCVar).T
YkIntOverC01Reshaped = {sp: YkIntOverC01[sp].reshape(nZTilde, nCTilde * nCVar).T for sp
    ↪ in species}
DkIntOverC01Reshaped = {sp: DkIntOverC01[sp].reshape(nZTilde, nCTilde * nCVar).T for sp
    ↪ in species}

# -- interpolate onto fine Z grid -----
W2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
T2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
dH2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
D2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
k2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
Cp2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
rho2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
HRR2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
GYcYc2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
GYcZ2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
GZYc2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
GZZ2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
sourceYc2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
sourceYcV2 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
sourceH22 = np.zeros((nCTilde * nCVar, nPOINTS_Z))
if NOEqn:
```

```

    sourceNO2 = np.zeros((nCTilde * nCVar, nPOINTS))
    Yk2 = {sp: np.zeros((nCTilde * nCVar, nPOINTS)) for sp in species}
    Dk2 = {sp: np.zeros((nCTilde * nCVar, nPOINTS)) for sp in species}

    for i in range(nCTilde * nCVar):
        W2[i, :] = m.interpExt(Z01, ZTilde01, WIntOverC01Reshaped[i, :])
        T2[i, :] = m.interpExt(Z01, ZTilde01, TIntOverC01Reshaped[i, :])
        dH2[i, :] = m.interpExt(Z01, ZTilde01, dHIntOverC01Reshaped[i, :])
        D2[i, :] = m.interpExt(Z01, ZTilde01, DIntOverC01Reshaped[i, :])
        k2[i, :] = m.interpExt(Z01, ZTilde01, kIntOverC01Reshaped[i, :])
        Cp2[i, :] = m.interpExt(Z01, ZTilde01, CpIntOverC01Reshaped[i, :])
        HRR2[i, :] = m.interpExt(Z01, ZTilde01, HRRIntOverC01Reshaped[i, :])
        rho2[i, :] = m.interpExt(Z01, ZTilde01, rhoIntOverC01Reshaped[i, :])
        sourceYc2[i, :] = m.interpExt(Z01, ZTilde01, sourceYcIntOverC01Reshaped[i, :])
        sourceYcV2[i, :] = m.interpExt(Z01, ZTilde01, sourceYcVIntOverC01Reshaped[i, :])
        sourceH22[i, :] = m.interpExt(Z01, ZTilde01, sourceH2IntOverC01Reshaped[i, :])
        if NOEqn:
            sourceNO2[i, :] = m.interpExt(Z01, ZTilde01, sourceNOIntOverC01Reshaped[i, :])
        GYcYc2[i, :] = m.interpExt(Z01, ZTilde01, GYcYcIntOverC01Reshaped[i, :])
        GYcZ2[i, :] = m.interpExt(Z01, ZTilde01, GYcZIntOverC01Reshaped[i, :])
        GZYc2[i, :] = m.interpExt(Z01, ZTilde01, GZYcIntOverC01Reshaped[i, :])
        GZZ2[i, :] = m.interpExt(Z01, ZTilde01, GZZIntOverC01Reshaped[i, :])
        for sp in species:
            Yk2[sp][i, :] = m.interpExt(Z01, ZTilde01, YkIntOverC01Reshaped[sp][i, :])
            Dk2[sp][i, :] = m.interpExt(Z01, ZTilde01, DkIntOverC01Reshaped[sp][i, :])

    W1 = W2.T.reshape(nPOINTS, nCTilde, nCVar)
    T1 = T2.T.reshape(nPOINTS, nCTilde, nCVar)
    dH1 = dH2.T.reshape(nPOINTS, nCTilde, nCVar)
    k1 = k2.T.reshape(nPOINTS, nCTilde, nCVar)
    Cp1 = Cp2.T.reshape(nPOINTS, nCTilde, nCVar)
    rho1 = rho2.T.reshape(nPOINTS, nCTilde, nCVar)
    D1 = D2.T.reshape(nPOINTS, nCTilde, nCVar)
    HRR1 = HRR2.T.reshape(nPOINTS, nCTilde, nCVar)
    GYcYc1 = GYcYc2.T.reshape(nPOINTS, nCTilde, nCVar)
    GYcZ1 = GYcZ2.T.reshape(nPOINTS, nCTilde, nCVar)
    GZYc1 = GZYc2.T.reshape(nPOINTS, nCTilde, nCVar)
    GZZ1 = GZZ2.T.reshape(nPOINTS, nCTilde, nCVar)
    sourceYc1 = sourceYc2.T.reshape(nPOINTS, nCTilde, nCVar)
    sourceYcV1 = sourceYcV2.T.reshape(nPOINTS, nCTilde, nCVar)
    sourceH21 = sourceH22.T.reshape(nPOINTS, nCTilde, nCVar)
    if NOEqn:
        sourceNO1 = sourceNO2.T.reshape(nPOINTS, nCTilde, nCVar)
    Yk1 = {sp: Yk2[sp].T.reshape(nPOINTS, nCTilde, nCVar) for sp in species}
    Dk1 = {sp: Dk2[sp].T.reshape(nPOINTS, nCTilde, nCVar) for sp in species}

```

As previously done, the `ZVar` matrix is constructed together with the parameters of the β -PDF function. The corresponding derivatives are also computed in order to perform integration by parts using the cumulative distribution function.

```

# Z variance grid
iexp      = np.arange(1, nZVar + 1, 1)
ZVarMax01 = 0.25 * ((iexp - 1) / (nZVar - 1)) ** 2.5
[ZVAR01, ZTILDE01] = np.meshgrid(ZVarMax01, ZTilde01)
ZTILDE = ZTILDE01[1:-1, 1:-1]
ZVAR   = ZVAR01[1:-1, 1:-1]
ZVAR   = np.minimum(ZTILDE * (1 - ZTILDE), ZVAR)
idx     = ZVAR == ZTILDE * (1 - ZTILDE)
ZVAR[idx] = ZVAR[idx] - 1e-6

ZZ_3d = np.tile(Z01[:, None, None], (1, nCTilde, nCVar))
dZ     = np.diff(Z)

a_Z = ZTILDE * (ZTILDE * (1 - ZTILDE) / ZVAR - 1)
b_Z = (1 - ZTILDE) * (ZTILDE * (1 - ZTILDE) / ZVAR - 1)

# Savitzky-Golay derivatives over Z
dWdZ      = m.SavitzkyGolay(W1,      ZZ_3d, 0, 2, 1)
ddHdZ     = m.SavitzkyGolay(dH1,     ZZ_3d, 0, 2, 1)
dCpdZ     = m.SavitzkyGolay(Cp1,     ZZ_3d, 0, 2, 1)
dTdZ      = m.SavitzkyGolay(T1,      ZZ_3d, 0, 2, 1)
dDdZ      = m.SavitzkyGolay(D1,      ZZ_3d, 0, 2, 1)
dkdZ      = m.SavitzkyGolay(k1,      ZZ_3d, 0, 2, 1)
drhodZ    = m.SavitzkyGolay(rho1,    ZZ_3d, 0, 2, 1)
dHRRdZ    = m.SavitzkyGolay(HRR1,    ZZ_3d, 0, 2, 1)
dGYcYcdZ  = m.SavitzkyGolay(GYcYc1,  ZZ_3d, 0, 2, 1)
dGYcZdZ   = m.SavitzkyGolay(GYcZ1,   ZZ_3d, 0, 2, 1)
dGZYcdZ   = m.SavitzkyGolay(GZYc1,   ZZ_3d, 0, 2, 1)
dGZZdZ    = m.SavitzkyGolay(GZZ1,    ZZ_3d, 0, 2, 1)
dsourceYcdZ = m.SavitzkyGolay(sourceYc1, ZZ_3d, 0, 2, 1)
dsourceYcVdZ = m.SavitzkyGolay(sourceYcV1, ZZ_3d, 0, 2, 1)
dsourceH2dZ = m.SavitzkyGolay(sourceH21, ZZ_3d, 0, 2, 1)
if NOEqn:
    dsourceN0dZ = m.SavitzkyGolay(sourceN01, ZZ_3d, 0, 2, 1)
dYkdZ = {sp: m.SavitzkyGolay(Yk1[sp], ZZ_3d, 0, 2, 1) for sp in species}
dDkdZ = {sp: m.SavitzkyGolay(Dk1[sp], ZZ_3d, 0, 2, 1) for sp in species}

```

As described for the integration over C , the endpoints are excluded during the integration stage, which is again performed in parallel.

```

# -----
# PARALLEL BETA-PDF INTEGRATION OVER Z
# -----
shared_arrZ = {
    'Z01': Z01,
    'W1': W1, 'dWdZ': dWdZ,
    'dH1': dH1, 'ddHdZ': ddHdZ,
    'T1': T1, 'dTdZ': dTdZ,
    'Cp1': Cp1, 'dCpdZ': dCpdZ,
    'D1': D1, 'dDdZ': dDdZ,
    'k1': k1, 'dkdZ': dkdZ,

```

```

    'rho1': rho1, 'drhodZ': drhodZ,
    'HRR1': HRR1, 'dHRRdZ': dHRRdZ,
    'GYcYc1': GYcYc1, 'dGYcYcdZ': dGYcYcdZ,
    'GYcZ1': GYcZ1, 'dGYcZdZ': dGYcZdZ,
    'GZYc1': GZYc1, 'dGZYcdZ': dGZYcdZ,
    'GZZ1': GZZ1, 'dGZZdZ': dGZZdZ,
    'sourceYc1': sourceYc1, 'dsourceYcdZ': dsourceYcdZ,
    'sourceYcV1': sourceYcV1, 'dsourceYcVdZ': dsourceYcVdZ,
    'sourceH21': sourceH21, 'dsourceH2dZ': dsourceH2dZ,
    'Yk1': Yk1, 'dYkdZ': dYkdZ,
    'Dk1': Dk1, 'dDkdZ': dDkdZ,
}
if NOEqn:
    shared_arrZ['sourceN01'] = sourceN01
    shared_arrZ['dsourceN0dZ'] = dsourceN0dZ

tasks_Z = [
    (i, j, float(a_Z[i, j]), float(b_Z[i, j]))
    for i in range(nZTilde - 2)
    for j in range(nZVar - 2)
]

print(f"\nBeta-PDF integration over Z - {len(tasks_Z)} tasks on {N_WORKERS} workers...")

counter_Z = Value(ctypes.c_int, 0)

WIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
dHIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
TIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
CpIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
rhoIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
DIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
kIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
HRRIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
GYcYcIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
GYcZIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
GZYcIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
GZZIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
sourceYcIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
sourceYcVIntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
sourceH2IntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
if NOEqn:
    sourceN0IntOverZ = np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar))
YkIntOverZ = {sp: np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar)) for sp in species}
DkIntOverZ = {sp: np.zeros((nZTilde - 2, nZVar - 2, nCTilde, nCVar)) for sp in species}

resZ_list = []
with Pool(processes=N_WORKERS,
          initializer=_init_intZ,
          initargs=(shared_arrZ, species, NOEqn, counter_Z, len(tasks_Z))) as pool:
    for res in pool.imap_unordered(_worker_intZ, tasks_Z):
        resZ_list.append(res)

```

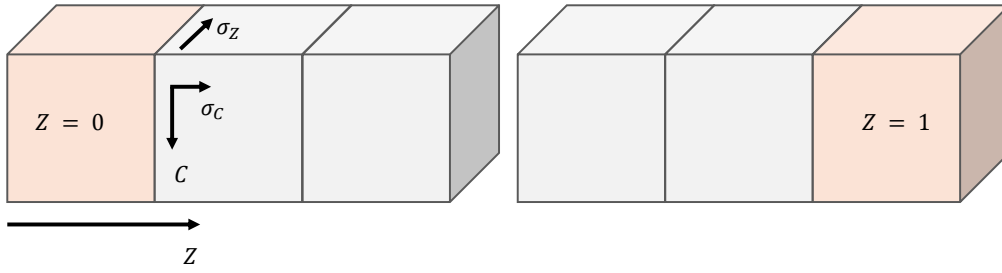


Figure 4. 4-D matrix with extrema at $Z = 0$, $Z = 1$, $\sigma_Z = 0$ and $\sigma_Z = 0.25$ excluded.

```

for res in resZ_list:
    i, j = res['i'], res['j']
    WIntOverZ[i, j] = res['W']
    dHIntOverZ[i, j] = res['dH']
    TIntOverZ[i, j] = res['T']
    CpIntOverZ[i, j] = res['Cp']
    DIntOverZ[i, j] = res['D']
    kIntOverZ[i, j] = res['k']
    rhoIntOverZ[i, j] = res['rho']
    HRRIntOverZ[i, j] = res['HRR']
    GYcYcIntOverZ[i, j] = res['GYcYc']
    GYcZIntOverZ[i, j] = res['GYcZ']
    GZYcIntOverZ[i, j] = res['GZYc']
    GZZIntOverZ[i, j] = res['GZZ']
    sourceYcIntOverZ[i, j] = res['sourceYc']
    sourceYcVIntOverZ[i, j] = res['sourceYcV']
    sourceH2IntOverZ[i, j] = res['sourceH2']
    if NOEqn:
        sourceNOIntOverZ[i, j] = res['sourceNO']
    for sp in species:
        YkIntOverZ[sp][i, j] = res['Yk'][sp]
        DkIntOverZ[sp][i, j] = res['Dk'][sp]

```

The data are now organized as shown in Fig. 4

Extremal values corresponding to $Z = 0$, $Z = 1$, $\sigma_Z = 0$, and $\sigma_Z = 0.25$ are then added manually using the `addBoundries4D` function.

```

# -- Add Z-boundaries -----
TIntOverZ01 = m.addBoundries4D(TIntOverZ,      TIntOverC01,      TTable,
    ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
dHIntOverZ01 = m.addBoundries4D(dHIntOverZ,    dHIntOverC01,    dHTable,
    ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
WIntOverZ01 = m.addBoundries4D(WIntOverZ,      WIntOverC01,      MMWTable,
    ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
DIntOverZ01 = m.addBoundries4D(DIntOverZ,      DIntOverC01,      DTable,
    ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
kIntOverZ01 = m.addBoundries4D(kIntOverZ,      kIntOverC01,      kTable,
    ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)

```

```

CpIntOverZ01      = m.addBoundaries4D(CpIntOverZ,      CpIntOverC01,      CpTable,
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
rhoIntOverZ01     = m.addBoundaries4D(rhoIntOverZ,     rhoIntOverC01,     rhoTable,
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
HRRIntOverZ01     = m.addBoundaries4D(HRRIntOverZ,     HRRIntOverC01,     HRRTable,
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
sourceYcIntOverZ01 = m.addBoundaries4D(sourceYcIntOverZ, sourceYcIntOverC01,
  ↳ omegakTable['H2O'], CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
sourceYcVIntOverZ01 =
  ↳ m.addBoundaries4D(sourceYcVIntOverZ, sourceYcVIntOverC01, sourceYcVTable, CTable,
  ↳ CTilde01, nZTilde, nCTilde, nCVar, nZVar)
sourceH2IntOverZ01 = m.addBoundaries4D(sourceH2IntOverZ, sourceH2IntOverC01,
  ↳ omegakTable['H2'], CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
if NOEqn:
    sourceNOIntOverZ01 = m.addBoundaries4D(sourceNOIntOverZ, sourceNOIntOverC01,
      ↳ omegakTable['NO'], CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
GYcYcIntOverZ01    = m.addBoundaries4D(GYcYcIntOverZ,    GYcYcIntOverC01,    GYcYcTable,
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
GYcZIntOverZ01     = m.addBoundaries4D(GYcZIntOverZ,     GYcZIntOverC01,     GYcZTable,
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
GZYcIntOverZ01     = m.addBoundaries4D(GZYcIntOverZ,     GZYcIntOverC01,     GZYcTable,
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
GZZIntOverZ01      = m.addBoundaries4D(GZZIntOverZ,      GZZIntOverC01,      GZZTable,
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar)
YkIntOverZ01 = {sp: m.addBoundaries4D(YkIntOverZ[sp], YkIntOverC01[sp], YkTable[sp],
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar) for sp in species}
DkIntOverZ01 = {sp: m.addBoundaries4D(DkIntOverZ[sp], DkIntOverC01[sp], DkTable[sp],
  ↳ CTable, CTilde01, nZTilde, nCTilde, nCVar, nZVar) for sp in species}

print("Added boundaries for Z = 0, Z = 1, sigma_Z = 0, sigma_Z = sigma_Z_MAX!")

```

The OpenFOAM code performs the interpolation in the order Z, C, σ_Z, σ_C ; therefore, the matrices are permuted to match this ordering and reshaped into a one-dimensional vector.

```

# -- Permute (Z, C, Cv, Zv)
↳ -----
TIntOverZ01T      = TIntOverZ01.transpose(0, 2, 3, 1)
dHIntOverZ01T     = dHIntOverZ01.transpose(0, 2, 3, 1)
DIntOverZ01T      = DIntOverZ01.transpose(0, 2, 3, 1)
WIntOverZ01T      = WIntOverZ01.transpose(0, 2, 3, 1)
CpIntOverZ01T     = CpIntOverZ01.transpose(0, 2, 3, 1)
kIntOverZ01T      = kIntOverZ01.transpose(0, 2, 3, 1)
rhoIntOverZ01T    = rhoIntOverZ01.transpose(0, 2, 3, 1)
HRRIntOverZ01T    = HRRIntOverZ01.transpose(0, 2, 3, 1)
sourceYcIntOverZ01T = sourceYcIntOverZ01.transpose(0, 2, 3, 1)
sourceYcVIntOverZ01T = sourceYcVIntOverZ01.transpose(0, 2, 3, 1)
sourceH2IntOverZ01T = sourceH2IntOverZ01.transpose(0, 2, 3, 1)
if NOEqn:
    sourceNOIntOverZ01T = sourceNOIntOverZ01.transpose(0, 2, 3, 1)
GYcYcIntOverZ01T   = GYcYcIntOverZ01.transpose(0, 2, 3, 1)
GYcZIntOverZ01T    = GYcZIntOverZ01.transpose(0, 2, 3, 1)

```

```

GZYcIntOverZ01T      = GZYcIntOverZ01.transpose(0, 2, 3, 1)
GZZIntOverZ01T      = GZZIntOverZ01.transpose(0, 2, 3, 1)
YkIntOverZ01T = {sp: YkIntOverZ01[sp].transpose(0, 2, 3, 1) for sp in species}
DkIntOverZ01T = {sp: DkIntOverZ01[sp].transpose(0, 2, 3, 1) for sp in species}

# -- Reshape for OpenFOAM -----
def _reshape_oF(arr):
    return arr.reshape(nZTilde, nCTilde, -1, order='F').reshape(-1, 1)

ToF      = _reshape_oF(TIntOverZ01T)
dHoF     = _reshape_oF(dHIntOverZ01T)
DoF      = _reshape_oF(DIntOverZ01T)
WoF      = _reshape_oF(WIntOverZ01T)
CpoF     = _reshape_oF(CpIntOverZ01T)
koF      = _reshape_oF(kIntOverZ01T)
rhooF    = _reshape_oF(rhoIntOverZ01T)
HRRoF    = _reshape_oF(HRRIntOverZ01T)
sourceYcoF = _reshape_oF(sourceYcIntOverZ01T)
sourceYcVoF = _reshape_oF(sourceYcVIntOverZ01T)
sourceH2oF = _reshape_oF(sourceH2IntOverZ01T)
if NOEqn:
    sourceN0oF = _reshape_oF(sourceN0IntOverZ01T)
GYcYcoF = _reshape_oF(GYcYcIntOverZ01T)
GYcZoF  = _reshape_oF(GYcZIntOverZ01T)
GZYcoF  = _reshape_oF(GZYcIntOverZ01T)
GZZoF   = _reshape_oF(GZZIntOverZ01T)
YkoF = {sp: _reshape_oF(YkIntOverZ01T[sp]) for sp in species}
DkoF = {sp: _reshape_oF(DkIntOverZ01T[sp]) for sp in species}

# -- Clip source terms -----
sourceYcoF[sourceYcoF < 0] = 0
sourceYcVoF[sourceYcoF < 0] = 0
if NOEqn:
    sourceN0oF[sourceN0oF < 0] = 0
for sp in species:
    YkoF[sp][YkoF[sp] < 0] = 0

ZVec      = ZTilde01
CVec      = CTilde01
ZVarVec   = ZVarMax01
CVarVec   = CVarMax01

# -- Export -----
if exportTable:
    print("Writing Table to file...\n")
    m.writeFGMTable4D(
        zLean, zRich, tin, p,
        Air["rhoAir"], Air["kAir"], Air["CpAir"], Air["WAir"], Air["DAir"],
        ↪ Air["D_NOAir"],
        Fuel["rhoFuel"], Fuel["kFuel"], Fuel["CpFuel"], Fuel["WFuel"], Fuel["DFuel"],
        ↪ Fuel["D_NOFuel"],
        ZTilde01, CTilde01, ZVarVec, CVarVec,
        sLVec.flatten(), tauVec.flatten(), lFVec.flatten(),

```

```

sourceH2oF.flatten(), sourceNOoF.flatten() if NOEqn else
↳ np.zeros_like(sourceH2oF.flatten()),
sourceYcoF.flatten(), sourceYcVoF.flatten(),
ToF.flatten(), rhooF.flatten(), DoF.flatten(),
DkoF['NO'].flatten(), koF.flatten(), CpoF.flatten(), WoF.flatten(),
HRRoF.flatten(), dHoF.flatten(),
YkoF['H'].flatten(), YkoF['H2'].flatten(), YkoF['H2O'].flatten(),
YkoF['H2O2'].flatten(), YkoF['HO2'].flatten(),
YkoF['O'].flatten(), YkoF['O2'].flatten(), YkoF['OH'].flatten(),
YkoF['N2'].flatten(), YkoF['NO'].flatten(),
GYcYcoF.flatten(), GYcZoF.flatten(), GZYcoF.flatten(), GZZoF.flatten(),
nZTilde, nCTilde, nZVar, nCVar,
nameMech, filename="fgmProperties"
)
print("Tables successfully written in --> fgmProperties\n")

print("\nDone.")

```

The final table is written to the file `fgmProperties` and is then ready for use.

REFERENCES

- [1] E. M. Fortes, E. J. Pérez-Sánchez, A. Both, T. Grenga, and D. Mira, “Analysis of thermodiffusive instabilities in hydrogen/air premixed flames using a tabulated flamelet model”, *International Journal of Hydrogen Energy*, vol. 137, pp. 247–259, 2025, ISSN: 0360-3199. DOI: <https://doi.org/10.1016/j.ijhydene.2025.04.338>.
- [2] E. Pérez-Sánchez, E. Fortes, and D. Mira, “Assessment of the flamelet generated manifold method with preferential diffusion modeling for partially premixed hydrogen flames”, *Combustion and Flame*, vol. 277, p. 114 141, 2025, ISSN: 0010-2180. DOI: <https://doi.org/10.1016/j.combustflame.2025.114141>.